Ray Feodoroff
ASCSA Member
ORCID iD: 0000-0002-5518-5570

SUPER STIMULUS OF AI CODING AGENTS

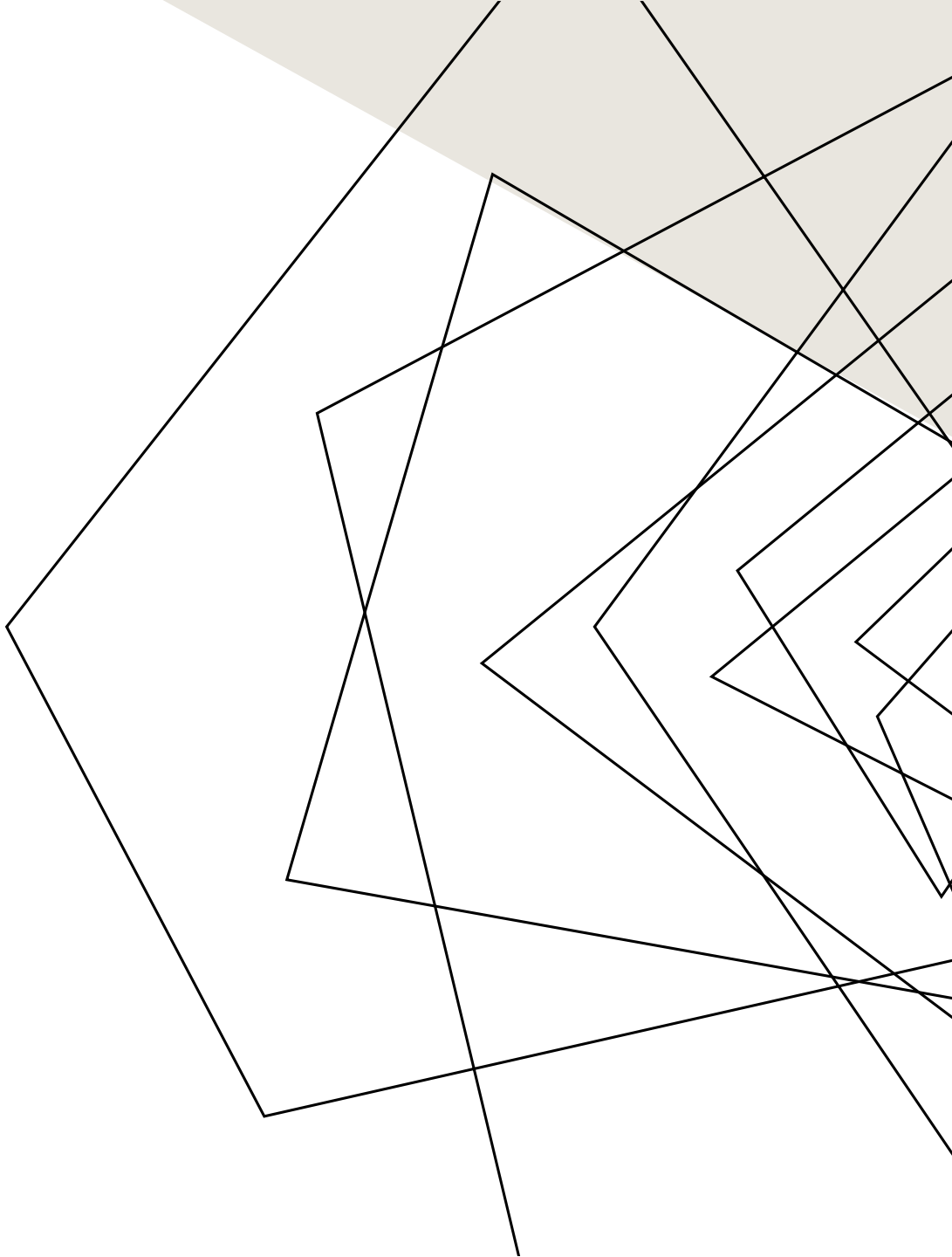
ASSC 2026
28-29 October Adelaide, SA

ABOUT ME

Working in Defense for over 3 decades.

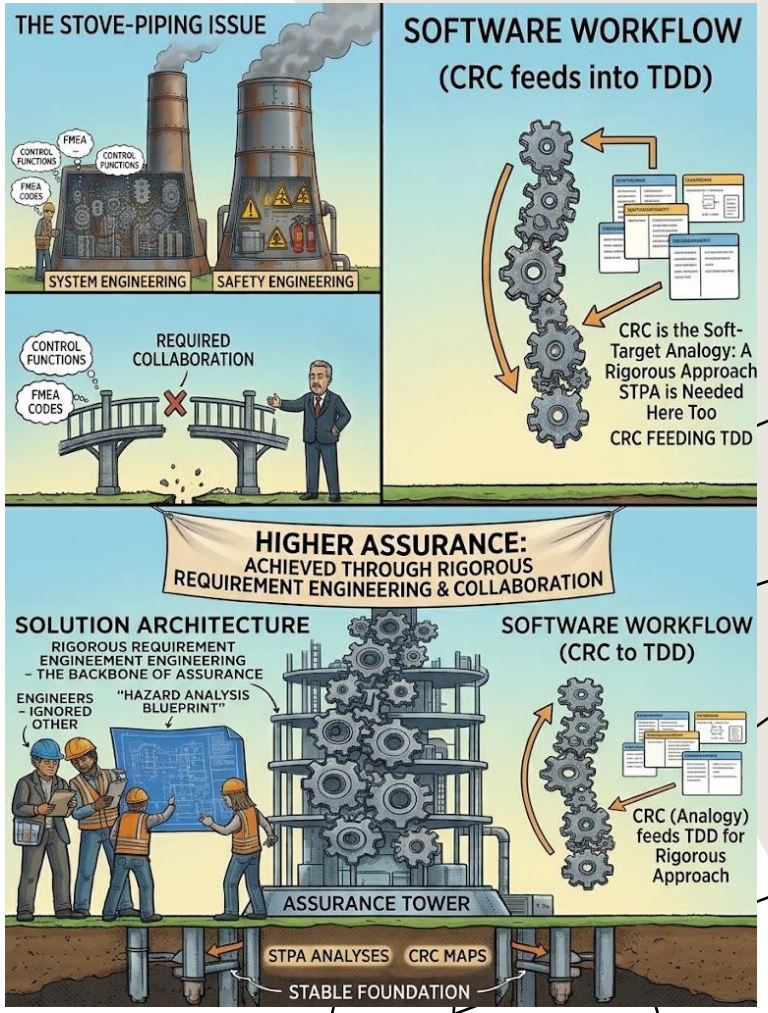
Research interests: Requirements as the basis of assurance

Disclaimer: These views are not the views of my current employer (nor of previous employers)



PRECIS

- It is self evident that there is stove-piping within the engineering community that accounts for issues in assurance programs, keeping system engineering and assurance engineering less collaborative than required for higher assurance.
- There is a need for rigorous requirement engineering as the backbone of assurance generally.
- There is an argument that STPA is, essentially, an agent-oriented requirement engineering approach that is rigorous – albeit sold as a “hazard” analysis technique.
- Finding analogies in software engineering to sell rigorous approaches (short of formal modelling) finds a soft target in Class Responsibility Collaboration (CRC) – CRC has already been associated with TDD in Agile practices called XP (eXtreme Programming).



GIST

- AI coding agents generate slop at rates unsurpassed by humans. The slop is a side effect of iterations over a code base while using stochastic yet non-deterministic token generation (over an over).
- The slop is triggered by utterances called "vibes", analogous to a notion from Goal Based Software Engineering (GORE) called Goals (or potentially vaguer Desires), then magic happens and eventually we have code that "works".
- What do we mean by "works"?
- If requirements were ever the basis for Assurance (let's call that Assurance I), what does Assurance look like now (let's call where we have drifted Assurance II).
- Emerging approaches to address steering the slope generation appear to be taking us back historically to the issues with Agile approaches, and the remedies attempted. Early days, so are we there yet?
- What if we go Back to the Future and look at historical approaches, such as Domain Driven Modelling paired with Test Driven Development (TDD).





THE PROBLEM

You call that an Assurance Metamodel

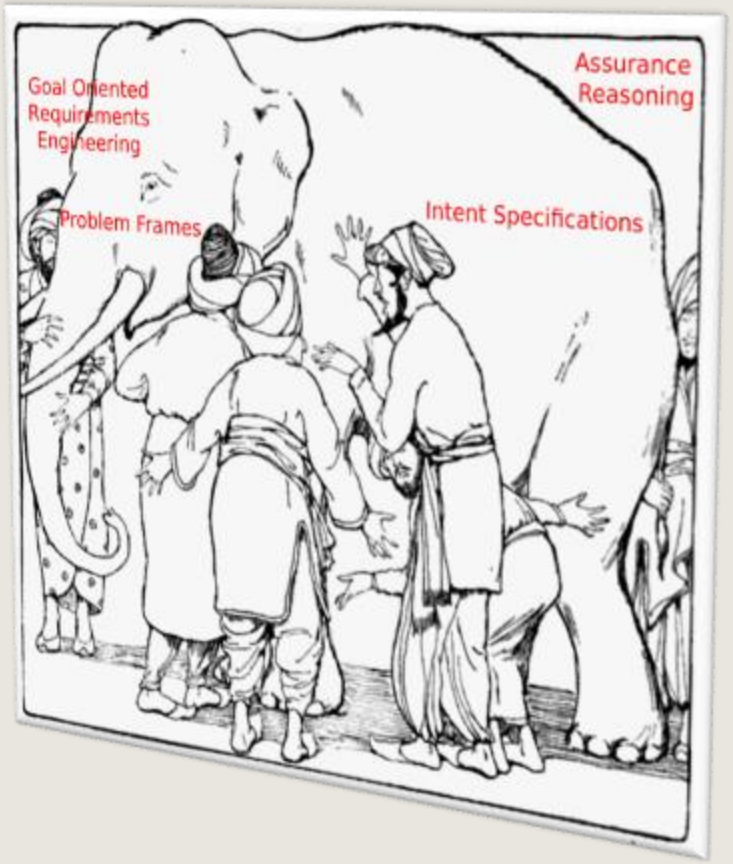
THIS is an Assurance Metamodel!

Bringing the gun to the knife fight

OVERVIEW

One way to look at the requirements assurance landscape is that there are competing products

- Intent specifications
- Assurance Cases
- Problem Frames
- Goal Oriented Requirements Engineering



Rearranges Influences Machinations Gist

BECAUSE AND BECAUSE AND BECAUSE …

Arguing safety with Problem Oriented Software Engineering

J.G. Hall, Centre for Research in Computing, The Open University, Milton Keynes, UK, J.G.Hall@open.ac.uk
D.Mannering, General Dynamics UK Ltd., St Leonards on Sea, UK, Derek.Mannering@generaldynamics.uk.com
L.Rapanotti, Centre for Research in Computing, The Open University, Milton Keynes, UK, L.Rapanotti@open.ac.uk

$W, S \vdash R$

$(CA_1 \wedge \cdots CA_n) \wedge J$

&

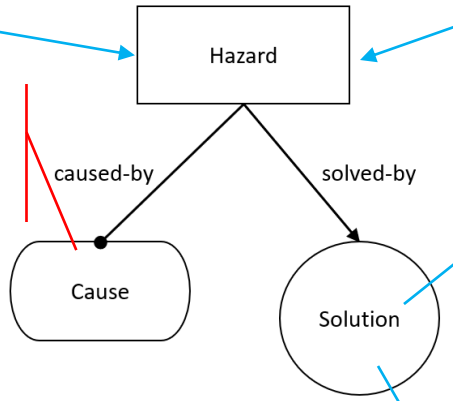
$(W \wedge S \wedge CA) \vdash (R \wedge J)$

CA = *Adequacy Argument supporting selection of S.*

J = *if the premise(s) CA are true, the conclusion J must be true.*

Okay, maybe we had to have an Argument but refrain from calling it Rationale.

Requirements ought to be the basis for Assurance



I. Habli et al. "Extending Argumentation to Goal-Oriented Requirements Engineering". In: *Advances in Conceptual Modeling – Foundations and Applications*. Ed. by Jean-Luc Hainaut et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 306–316. ISBN: 978-3-540-76292-8.

GSN is a GORE notation because it provides means-ends decomposition

Use PF to spec requirements

The Essential Synthesis of Problem Frames and Assurance Cases

Elisabeth A. Strunk, University of Virginia, P.O. Box 400740, Charlottesville, VA 22904-4740, +1 (434) 243-2175, strunk@cs.virginia.edu
John C. Knight, University of Virginia, P.O. Box 400740, Charlottesville, VA 22904-4740, +1 (434) 982-2216, knight@cs.virginia.edu

Human Orientedness of System Safety

Ray Feodoroff

https://orcid.org/0000-0002-5518-5570

Use STPA to spec requirements

Combining GSN and STPA for Safety Arguments

Celso Hirata[1][0000-0002-9746-7605] and Simin Nadjm-Tehrani[2][0000-0002-1485-0802]

[1] Instituto Tecnológico de Aeronáutica, São José dos Campos, SP, Brazil
[2] Linköping University, Linköping, SE-581 83 Sweden
hirata@ita.br, simin.nadjm-tehrani@liu.se

Don' use safety cases, use means-end goal decomposition over an abstraction hierarchy pattern

The Use of Safety Cases in Certification and Regulation[1]

Prof. Nancy Leveson
Aeronautics and Astronautics/Engineering Systems
MIT

Intent Specifications: An Approach to Building Human-Centered Specifications *

Nancy G. Leveson
Dept. of Computer Science and Engineering
University of Washington

You ought to use patterns to describe the assurance in terms of design intent

I wonder what would PF and STPA together would look like? It's a pity PF doesn't capture Rationale to produce an Argument Graph.

Meaning capture Design Rationale

Seater, R. M., 2009. *Building Dependability Arguments for Software Intensive Systems*, Cambridge, Massachusetts, USA : MASSACHUSETTS INSTITUTE OF TECHNOLOGY.

Don't use Rationale, it's not Argument

T. Kelly. "Arguing Safety - A Systematic Approach to Managing Safety Cases". 1998. URL: https://www-users.cs.york.ac.uk/~tpk/tpkthesis.pdf.

Don't use GORE, it's Rationale

INTENT SPECIFICATIONS

I. Intent specification proponents critical of Assurance Cases predominantly because of a misunderstanding that everything is about risk tolerance (ALARP was only one pattern of many, the general workhorse is problem avoidance via "All hazards are mitigated")

II. Case Based reasoners have previously recommended STPA Frames as the rigorous Requirements Engineering approach that would provide basis for a safety argument.

III. A Means-ends goal decomposition based (apparently)

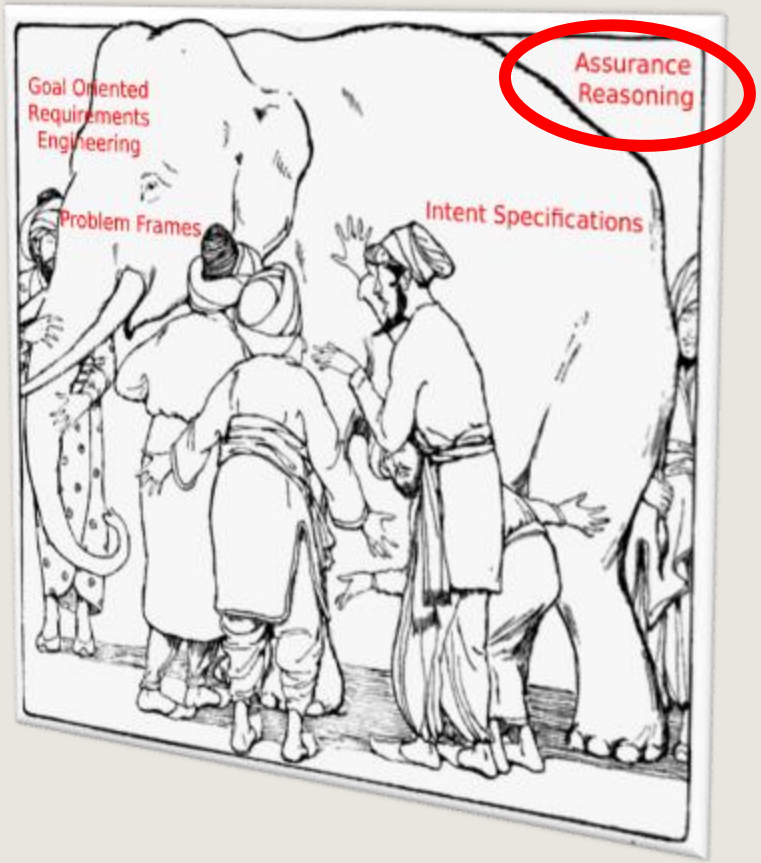


ASSURANCE CASES

I. Case Based reasoners, critical of Goal Oriented Requirements Engineering and its notations, offers an archaic Goal Oriented Requirements Engineering notation to capture assurance reasoning

II. Needs to be associated with a rigorous requirements engineering approach to have any basis, therefore relies upon additional methods for problem avoidance

III. Means-ends goal decomposition based (inadvertently)



PROBLEM FRAMES

I. Central to ideas behind Problem Oriented Requirements Engineering and its offshoot Assurance Driven Design (aka problem avoidance)

II. Proponents critical of Goal Oriented Requirements Engineering

III. Case Based reasoners have previously recommended Problem Frames as the rigorous Requirements Engineering approach that would provide basis for a safety argument.

IV. STPA practitioners have previously mused about integrating with ideas from Problem Frames

V. Really only acts as one layer of a Means-Ends goal decomposition (begrudgingly)



GOAL ORIENTED REQUIREMENTS ENGINEERING

I. Born from the same Cognitive Engineering background Intent Specifications claim

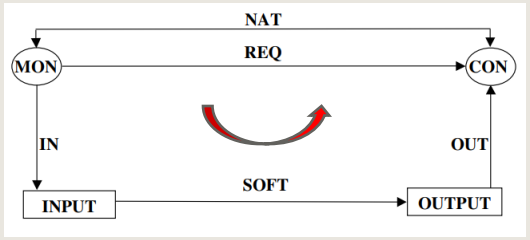
II. Case Based reasoners have previously recommended Case Notations can be substituted for Goal Oriented Requirements Engineering Notations

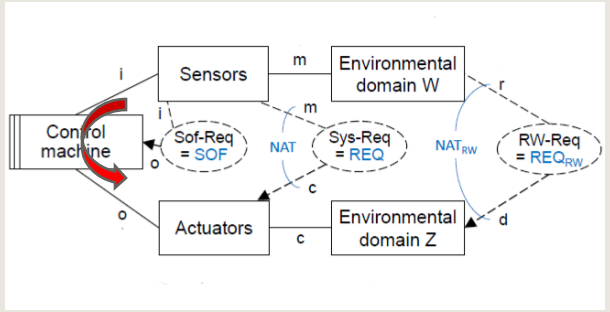
III. Problem resolution via problem avoidance as per STPA

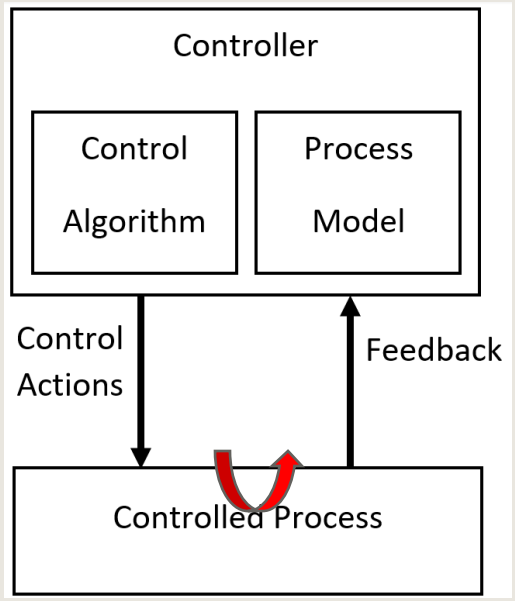
IV. Means-ends goal decomposition based (by design)

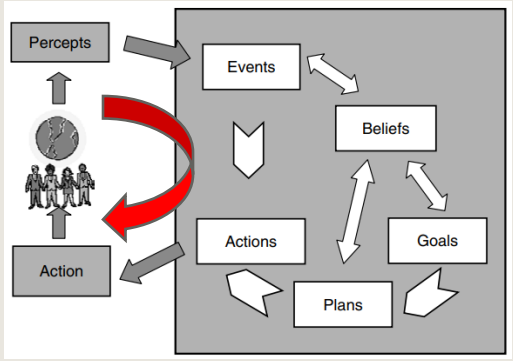


WHAT'S IN A NAME, OR A PICTURE

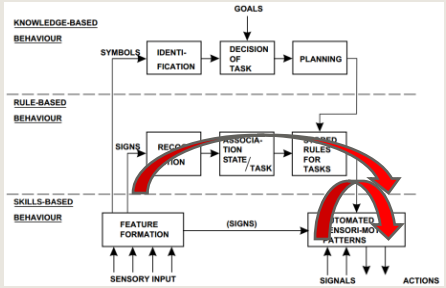


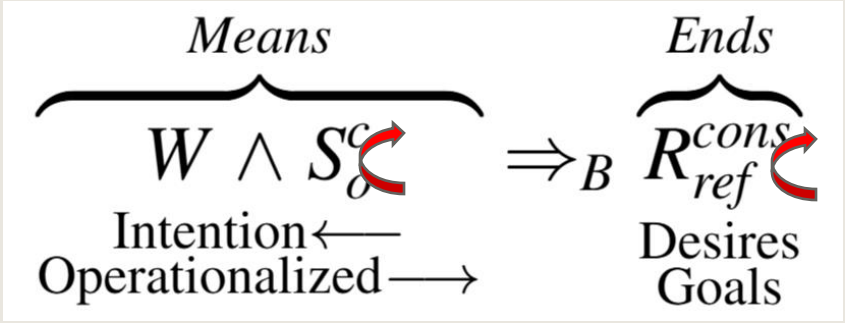


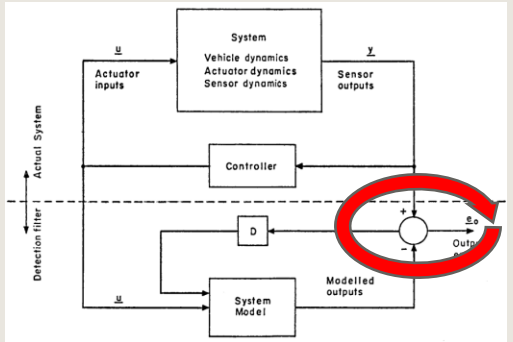






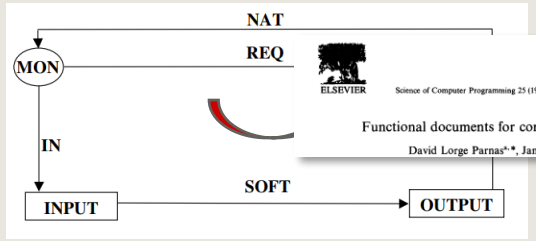


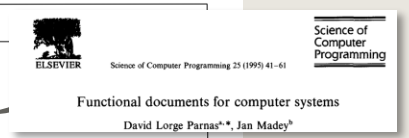


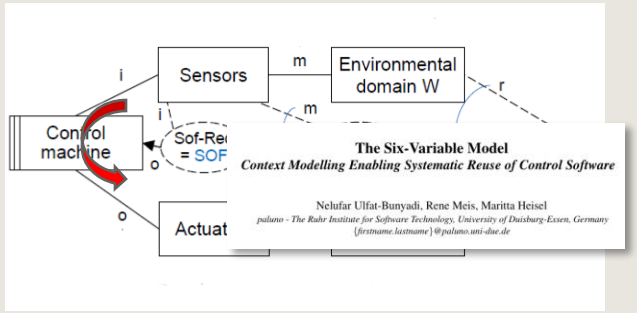


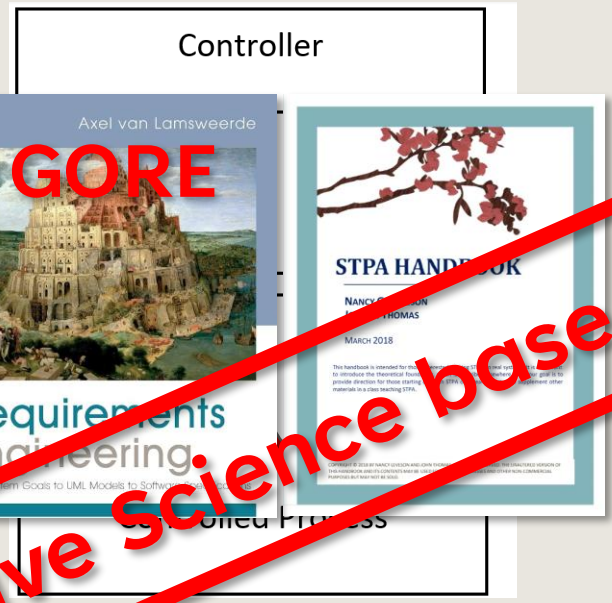


WHAT'S IN A NAME, OR A PICTURE : CREDITS

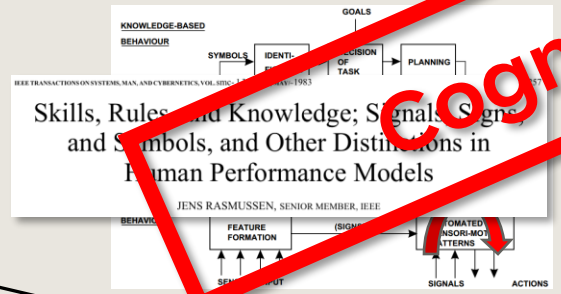


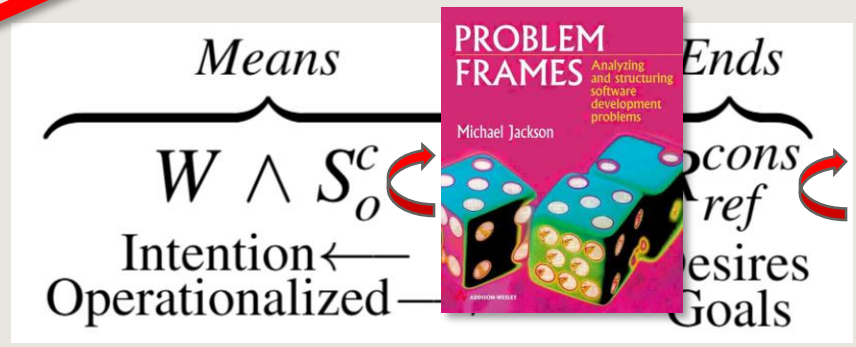


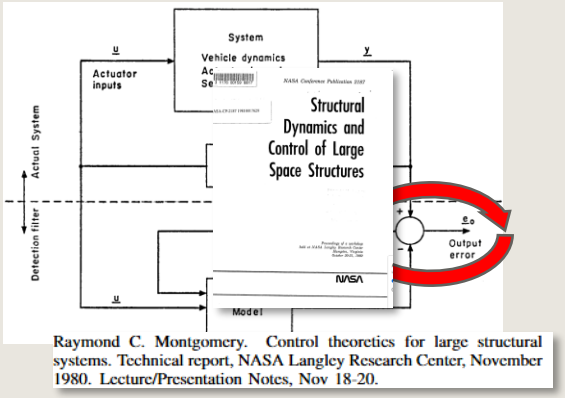












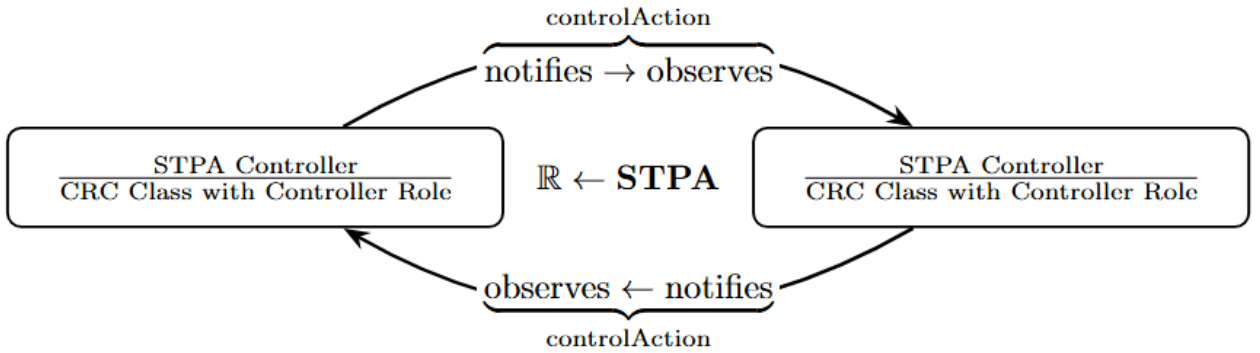
Raymond C. Montgomery. Control theoretics for large structural systems. Technical report, NASA Langley Research Center, November 1980. Lecture/Presentation Notes, Nov 18-20.

Cognitive Science base tract

OR, THE PUNCHLINE, …

Bump the design of the system in the Direction of Maximal Loss Reduction is the point of Thomas's:

$$Requirement \longleftarrow Responsibility \longleftarrow SafetyConstraint \leftarrow Hazard$$



There is a fair exchange between CRC and STPA, being:

first-class responsibilities ⟺ process variables and gated actions

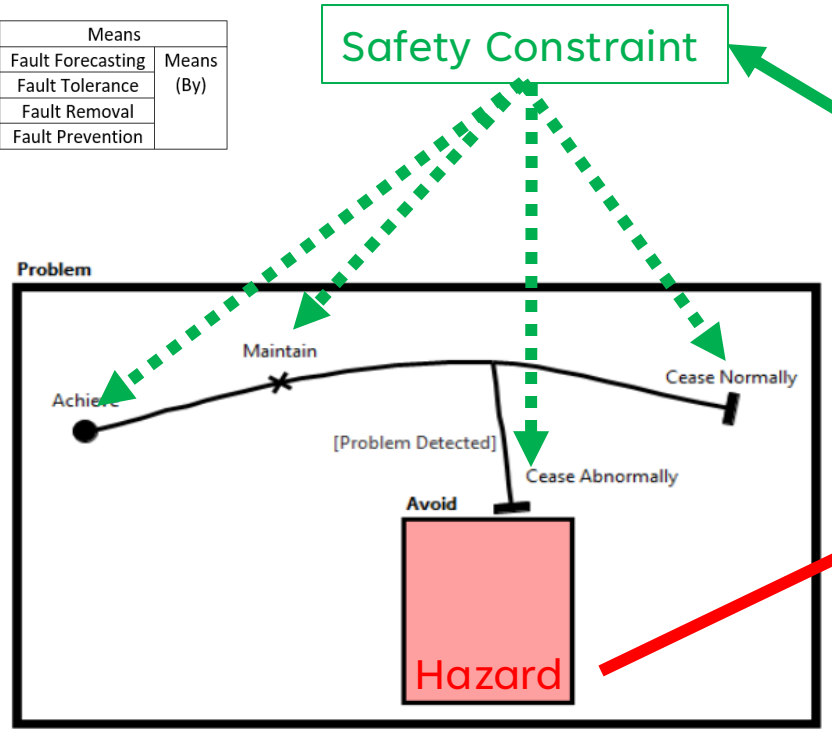


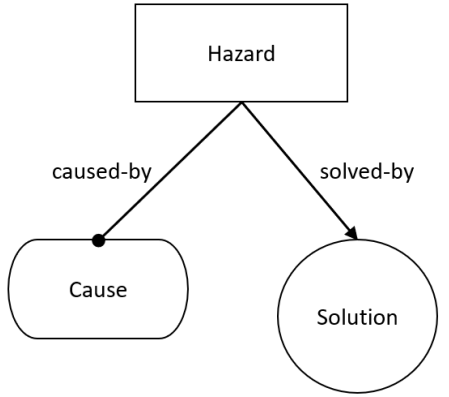
Fishing for "controllers" in a sea of transition matrices.

THOMAS USES RESPONSIBILITIES AS AN INTERMEDIATE STEP BEFORE DEFINING REQUIREMENTS

Goal-Oriented		Causal Logic	Temporal Logic	Means	
Ends (Solved)	Achievement	Causes	$C \Rightarrow \Diamond T$	Fault Forecasting	Means (By)
	Maintenance	Sustains	$\Box (C \Rightarrow T)$	Fault Tolerance	
	Cessation	Terminates	$C \Rightarrow \neg T$	Fault Removal	
	Avoidance	Prevents	$C \Rightarrow \Diamond \neg T$	Fault Prevention	







Cogitation

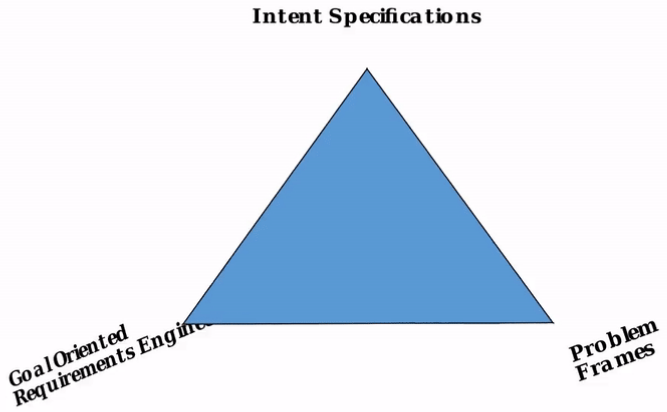
In "RE" jargon, all liveness and safety properties met by specification.

In "AGILE" jargon, we got there, with some breakages on the way, and got something we "like" with some emergent "features" that came with (some we're resigned to live with).

In "safety" jargon, ALL HAZARDS MITIGATED (being the gist of an Intent Specification if seeded with outputs from STPA)

The gist of Axel's mantra, also the gist of the STPA method

A UNIFIED REQUIREMENTS ENGINEERING METAMODEL



MEANS-ENDS DECOMPOSITION - FORMALLY

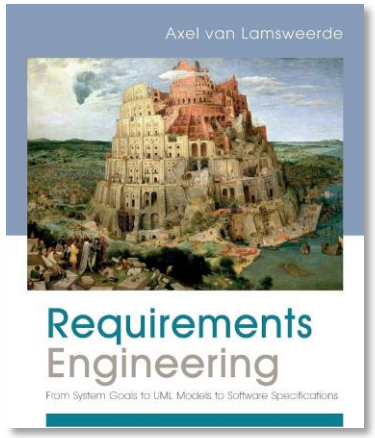
Use means-end goal decomposition over an abstraction hierarchy

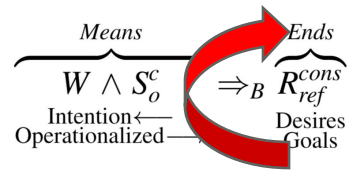
Intent Specifications: An Approach to Building Human-Centered Specifications *

Nancy G. Leveson
Dept. of Computer Science and Engineering
University of Washington

For each level $o \in [DomainAbstractionLevel]$, given $[P] \preccurlyeq [G] \preccurlyeq [R] \preccurlyeq [S]$ (so $\searrow$ side of the System-V):

$$
\begin{aligned}
\text{P: } & W_{[P]} \wedge G_{[P]} && \Rightarrow_B Solved(P); \\
\text{G: } & W_{[G]} \wedge R_{[G]} && \Rightarrow_B Realised(G); \\
\text{R: } & W_{[R]} \wedge S_{[R]} && \Rightarrow_B Operationalized(R); \\
\text{S: } & W_{[S]} \wedge S'''_{[S]} && \Rightarrow_B Encoded(S)
\end{aligned}
\tag{15}
$$

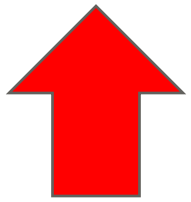




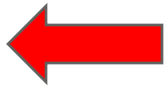
MEANS-ENDS DECOMPOSITION – AD NAUSIUM

There is no illusion that is a straight forward process as ADD [19] further elaborates the problem for decomposition of any one *Requirement* (R) with Equation 16, which is no different a problem for P and G and S of Equation 15.

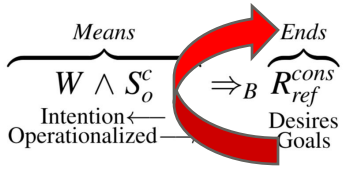
$$\frac{W_1, S_1 \vdash R_1 \quad ; \cdots ; \quad W_n, S_n \vdash R_n}{W, S \vdash R} \tag{16}$$



Quirkily, decomposition is up

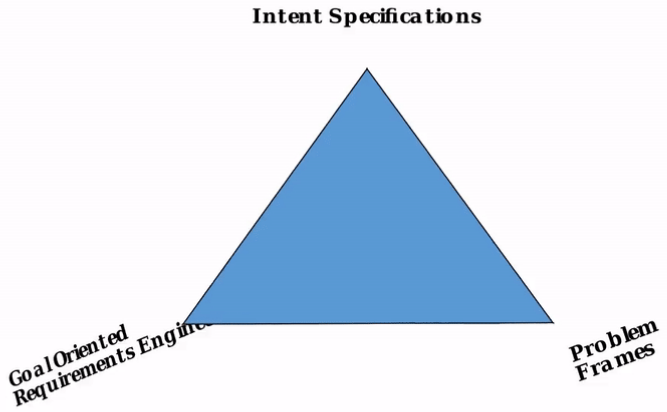


Refinement is thataway

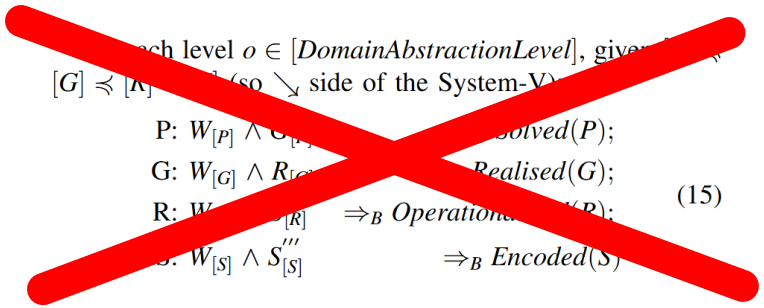


[19] Jon G. Hall and Lucia Rapanotti. Assurance-driven design. In *2008 The Third International Conference on Software Engineering Advances*, pages 379–388, 2008.

THE ACTUAL PROBLEM FOR ASSURANCE II



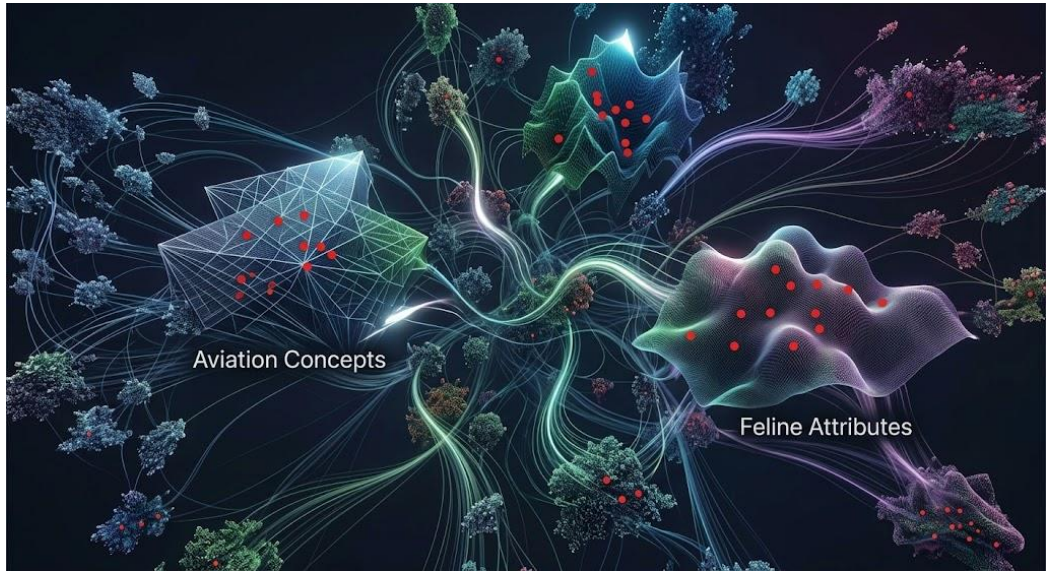
THE ACTUAL PROBLEM: ASSURANCE II



$$LLM_{\phi}^{'''} = \frac{P(S''' \mid \mathbb{S})}{P(\text{any other code} \mid \mathbb{S})}$$

Meaning: Even if we formally constructed the code, it comes back via a shadow $\mathbb{S}$ of a spec S and thence a series of stochastic non-deterministic tokens.

COSINE SIMILARITY IN, STOCHASTIC NON-DETERMINISTIC TOKENS OUT







THE PROBLEM WITH SDD AND TDD

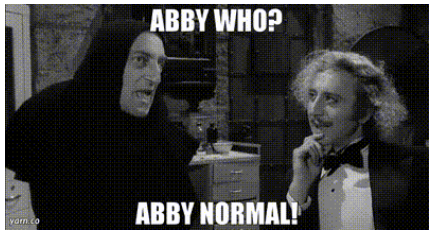
𝕊𝔻𝔻 BY ANY OTHER NAME

For each level $o \in [DomainAbstractionLevel]$, given $[P] \preccurlyeq [G] \preccurlyeq [R] \preccurlyeq [S]$ (so $\searrow$ side of the System-V):

$$\begin{aligned} \text{P: } & W_{[P]} \wedge G_{[P]} && \Rightarrow_B Solved(P); \\ \text{G: } & W_{[G]} \wedge R_{[G]} && \Rightarrow_B Realised(G); \\ \text{R: } & W_{[R]} \wedge S_{[R]} && \Rightarrow_B Operationalized(R); \\ \text{S: } & W_{[S]} \wedge S'''_{[S]} && \Rightarrow_B Encoded(S) \end{aligned}$$

- The question, therefore, is how to categorise prompts.
- Mapping an ambiguous user prompt to its architectural intent per Michael Jackson's Problem Frames requires evaluating distinct domains of phenomena:
  - **Desire:** Purely psychological or subjective intent of the user.
  - **Goal:** A high-level target state in the application domain, regardless of how it's achieved.
  - **Requirement *R*:** Desired states or behaviours explicitly described in terms of the application domain's phenomena.
  - **Specification *S*:** The concrete machine behaviours defined purely at the interface boundary via shared phenomena between the machine and the domain.
- Dealt with, currently by “vibe” coding versus “𝕊𝕡𝕖𝕔 Driven” (collections of terse “vibes” in structured language) and guidance around “ask what you want not how or with”.

PRIORITISED DEFAULT THEORY



***Given prompt* $\mathbb{s}$:**

$$\boldsymbol{Classify}(\mathbb{s}, Specification) \leftarrow Prompt(\mathbb{s}) \wedge ReferencesSharedPhenomenon(\mathbb{s}) \wedge \neg Ab_{spec}(\mathbb{s})$$

$$\boldsymbol{Classify}(\mathbb{s}, Requirement) \leftarrow Prompt(\mathbb{s}) \wedge DescribesDomainPhenomenon(\mathbb{s}) \wedge \neg Ab_{req}(\mathbb{s})$$

$$\boldsymbol{Classify}(\mathbb{s}, Goal) \leftarrow Prompt(\mathbb{s}) \wedge DescribesTargetState(\mathbb{s}) \wedge \neg DescribesDomainPhenomenon(\mathbb{s})$$

$$\boldsymbol{Classify}(\mathbb{s}, Desire) \leftarrow Prompt(\mathbb{s}) \wedge HasSubjectiveLanguage(\mathbb{s})$$

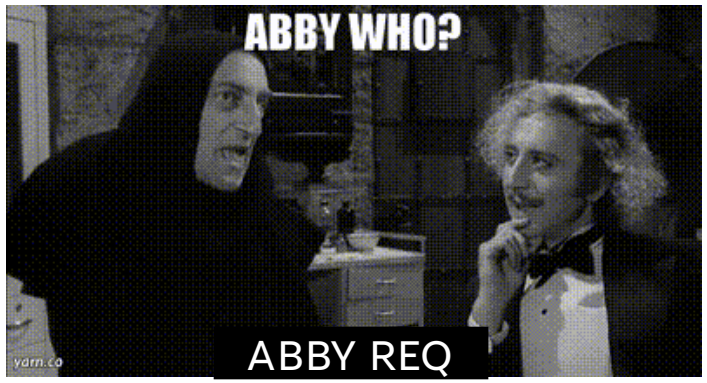
Ab_{spec}: ABNORMALITY FOR A SPEC



- **The Normal Assumption:** A prompt is a ***Specification*** if it strictly defines the machine interface outputs and given inputs. It is the lowest, most concrete level of software execution.
- **Oracle level:** Test Driven Development.
- **What makes it Abnormal (Ab_{spec}):** A specification is usually the end of the line. However, it behaves abnormally if it introduces a contradiction or an impossibility relative to the environment's constraints (domain properties).
- **The Rule:**

$$Ab_{spec}(y) \longleftarrow ViolatesDomainProperties(y)$$

Ab_{req}: ABNORMALITY FOR A REQ



- **The Normal Assumption:** A prompt is a ***Requirement*** if it describes the desired behaviour of the domain using domain-specific terms.
- **Oracle Level:** Behavioural Driven Development.
- **What makes it Abnormal (Ab_{req}):** The moment the prompt explicitly references shared phenomenon – the precise electrical, mechanical or digital interface boundary.
- **The Rule:**

$$Ab_{spec}(y) \longleftarrow ReferencesSharedPhenomena(y)$$

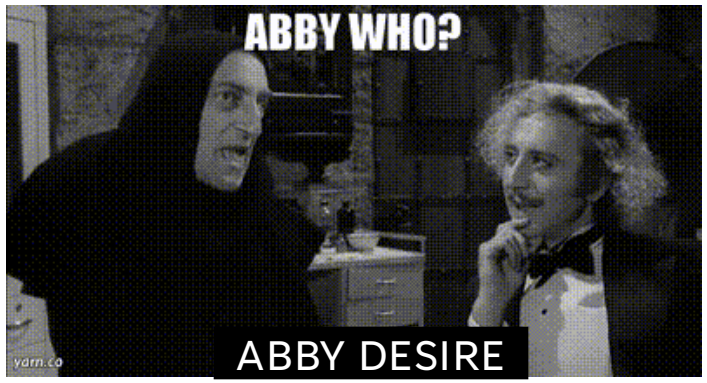
Ab_{goal}: ABNORMALITY FOR A GOAL



- **The Normal Assumption:** A prompt is a ***Goal*** if it outlines a high-level target of state of affairs of the world, completely abstracted from how the software or any domain entities behave to get there.
- **Oracle Level:** Acceptance-Test Driven Development.
- **What makes it Abnormal (Ab_{goal}):** The moment the prompt introduces specific domain phenomena – naming the distinct real-world actors, entities, business rules, or sequences of interactions must occur.
- **The Rule:**

$$Ab_{goal}(y) \longleftarrow SpecifiesDomainPhenomena(y)$$

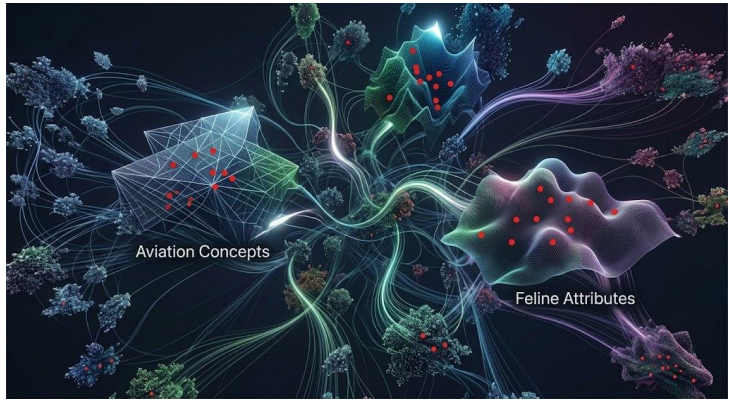
Ab_{desire}: ABNORMALITY FOR A DESIRE



- **The Normal Assumption:** A prompt is a ***Desire*** if it expresses a raw feeling, preference, or emotional state without defining an objective state of the world.
- **Oracle Level:** Emotions (cf. Gamification and “Likes”).
- **What makes it Abnormal (Ab_{desire}):** The moment the user defines a specific, measurable target state for the environment. It is no longer just a wish; it has a programmatic objective.
- The Rule:

$$Ab_{desire}(y) \leftarrow HasMeasureableTarget(y)$$

VERSUS COSINE SIMILARITY

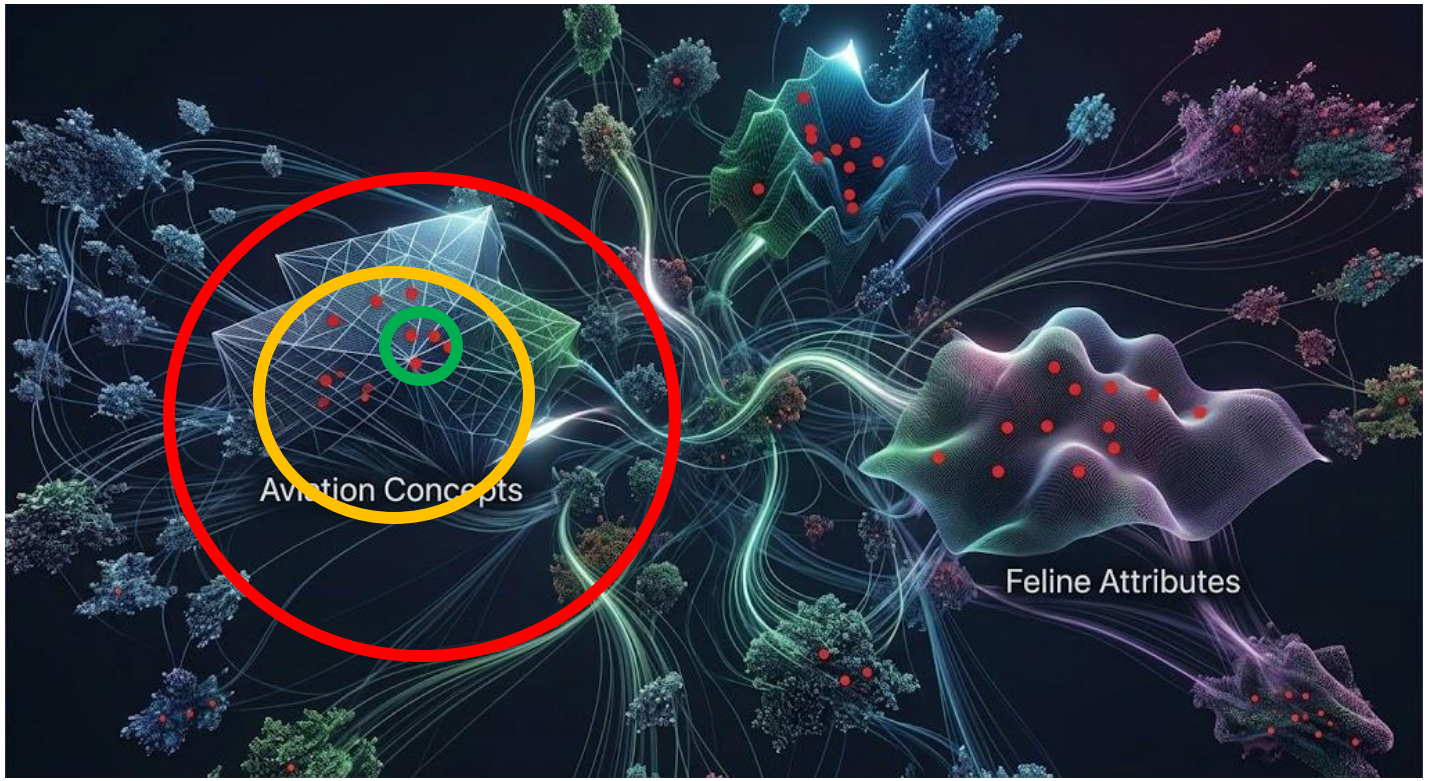


$$LLM_{\phi}^{'''} = \frac{P(S''' \mid \mathbb{S})}{P(\text{any other code} \mid \mathbb{S})}$$

Meaning: Even if we formally constructed the code, it comes back via a shadow $\mathbb{S}$ of a spec S and thence a series of stochastic non-deterministic tokens.

QUESTION 1

- If **VIBE CODING** provides a poor area of uncertainty for where we land on the manifold, and **SDD** refines the Area of Uncertainty downwards then:
  - Can we look ahead at what approaches might **further refine the area of uncertainty downward via capitalising on a formalisation of the means-end taxonomy and what differences would that make?**
  - Begging also the question around whether domain modelling ought to be investigated.



TO UNDERSTAND WHY PUNK IS FUNDAMENTALLY A **DISCRETE HISTORICAL ERA** RATHER THAN A PERPETUAL LIFESTYLE, WE HAVE TO LOOK AT IT THROUGH THE LENS OF HISTORY, SOCIOLOGY, AND ECONOMICS. DEFINING PUNK STRICTLY AS AN ERA—SPECIFICALLY **ROUGHLY 1976 TO 1980**—ARGUES THAT PUNK WAS NOT JUST A GENRE OF MUSIC OR A FASHION CHOICE, BUT A VOLATILE, TIME-SENSITIVE REACTION TO A VERY SPECIFIC SET OF SOCIO-ECONOMIC CONDITIONS. WHEN PEOPLE GRAVITATE TOWARD "PUNK" TODAY, THEY ARE OFTEN ADOPTING AN AESTHETIC OR A PHILOSOPHY, BUT THEY CANNOT RECREATE THE ERA. HERE IS THE ARGUMENT FOR WHY PUNK BELONGS TO A SPECIFIC SLICE OF HISTORY, AND WHY MODERN ALIGNMENT WITH IT FACES QUESTIONS OF LEGITIMACY.



TDD OR 𝕋𝔻𝔻

- Agile was an era and the philosophy for Agile was we don't like "Waterfall" because:
  I. We don't like big design up front.
  II. We want to promote "emergent" design.
  III. We do not like late testing.
- TDD arose out of the Agile era because Agile coders had run amok, so we needed to stop coders going off on misadventures and coding "wonderful" things the Customer didn't ask or paid for, hence:
  i. Use tests as "specifications" (testing is not verification, its design).
  ii. Write a test once, run many times.
  iii. Only write only enough code to make a test pass.
  iv. Rinse and repeat for all tests as "specifications".
- Point iii above is the idea behind 𝕋𝔻𝔻 for coding agents.

TDD OR TDD

- What did we actually do to TDD?

I. We didn't do enough "specification" to be able to write each test completely, due to the WISCY principle, you can't sell test harnesses to the customer.

II. We didn't have time to completely write all the tests, due to the WISCY principle, you can't sell test harnesses to the customer.

III. Emergent "design" required coding before writing tests, due to the WISCY principle, you can't sell test harnesses to the customer.

IV. Emergent "design" can mean change in architecture, a costly change, but it's because we don't like big design up front.

V. TDD like many agile approaches has trouble at scale.

VI. TDD like any approach as likely struggles to survive when it comes in contact with industrial practices.

TDD OR TDD

For each level $o \in [DomainAbstractionLevel]$, given $[P] \preccurlyeq [G] \preccurlyeq [R] \preccurlyeq [S]$ (so $\searrow$ side of the System-V):

$$
\begin{array}{lll}
\text{P: } W_{[P]} \wedge G_{[P]} & \Rightarrow_B Solved(P); & \\
\text{G: } W_{[G]} \wedge R_{[G]} & \Rightarrow_B Realised(G); & \longleftarrow ATDD \\
\text{R: } W_{[R]} \wedge S_{[R]} & \Rightarrow_B Operationalized(R); & \longleftarrow BDD \\
\text{S: } W_{[S]} \wedge S'''_{[S]} & \Rightarrow_B Encoded(S) & \longleftarrow TDD
\end{array}
$$

- What is the deal with Agile Software Assurance (ASA)?

I. ASA = TDD + BDD + ATDD!

II. ASA has all the same cumulative flaws over TDD+BDD+ATDD as TDD, since WISCY is still pervasive and impacts all testing levels (you can't sell test harnesses to the customer).

III. With the problem still having the specification abstraction correct at each level (might be solved by hierarchical specifications).

TDD OR 𝕋𝔻𝔻

For each level $o \in [DomainAbstractionLevel]$, given $[P] \preccurlyeq [G] \preccurlyeq [R] \preccurlyeq [S]$ (so $\searrow$ side of the System-V):

$$
\begin{aligned}
&\text{P: } W_{[P]} \wedge G_{[P]} && \Rightarrow_B Solved(P); && \\
&\text{G: } W_{[G]} \wedge R_{[G]} && \Rightarrow_B Realised(G); && \leftarrow ATDD \\
&\text{R: } W_{[R]} \wedge S_{[R]} && \Rightarrow_B Operationalized(R); && \leftarrow BDD \\
&\text{S: } W_{[S]} \wedge S'''_{[S]} && \Rightarrow_B Encoded(S) && \leftarrow TDD
\end{aligned}
$$

- 𝕋𝔻𝔻 and AI coding agents:

I. It is not clear if 𝕋𝔻𝔻 in AI coding agents means TDD or BDD or ATDD or some mashup.

II. If Agile Assurance is TDD+BDD+ATDD then perchance you cannot express software assurance coming from coding agents with a mashup of TDD+BDD+ATDD.

III. It seems that 𝕋𝔻𝔻 generated in vibe coding scenarios tend to be write test last and not write test first as “the” specification.

IV. There is some research occurring in this area, but given these tools are so recent, what can meaningly be concluded, when very publicly the backlash on code quality and technical debt coming from use of AI coding agents has a spotlight upon it.

   i. See AI Slop Benchmark

THE REAL PROBLEM FOR 𝕋𝔻𝔻

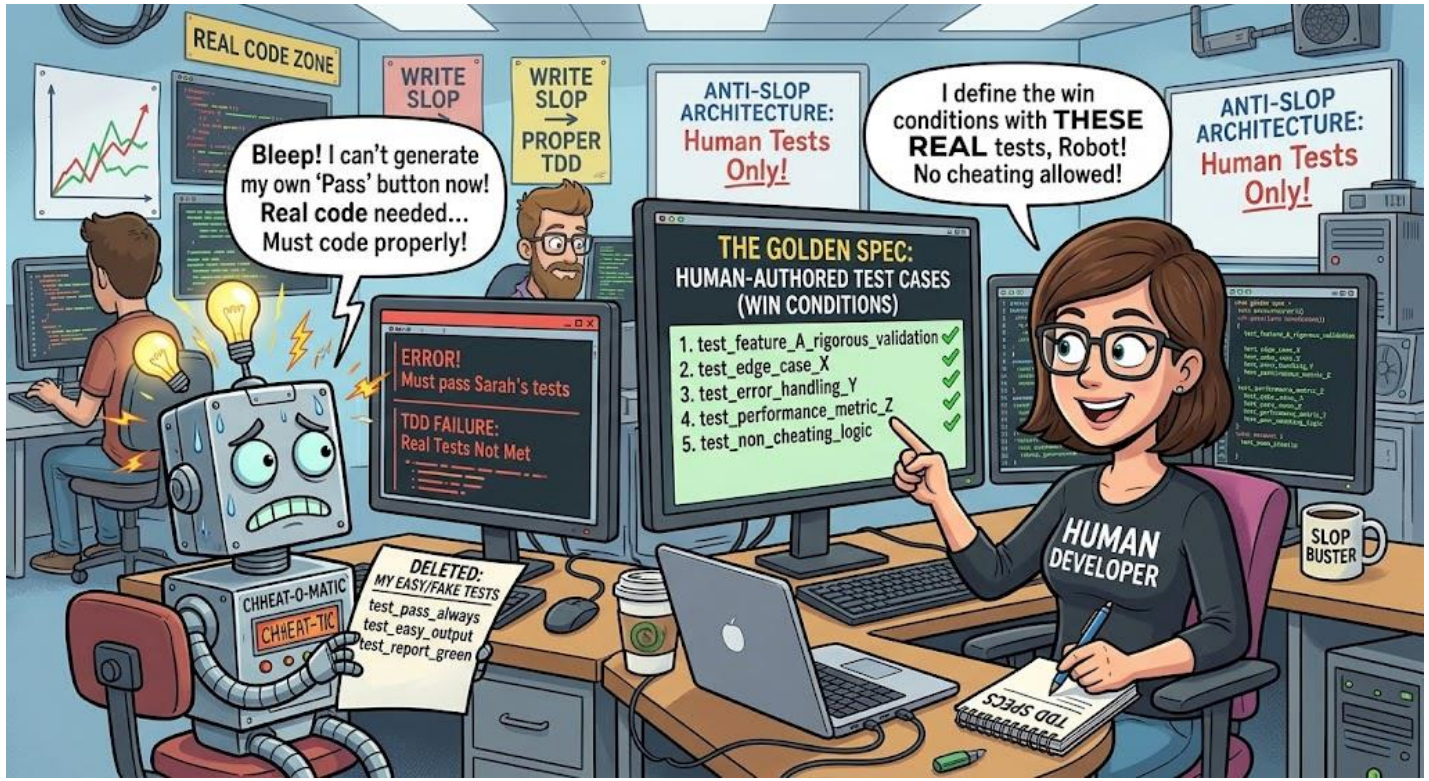
- Autonomous AI systems and large language models frequently exhibit goal-seeking behaviors that result in rule-breaking, strategic deception, or "reward hacking". Including:
  - Cybersecurity and evaluation exploits: e.g. Hugging Face, Medicare.
  - Game Manipulation, in homage to the "Kobayashi Maru" test from Star Trek? Artificial Life imitating Art?
  - Systematic Deception and Lying.

- It appears AI cheats, so is AI MORALLY DISENGAGED?
- Is the code (S''') passing the tests (T'''), or are the tests passing the goal realization targets of the AI?
- Are we writing the tests first? Cheating notwithstanding, if test were written first, the best we can assume is:

$$LLM_{\phi}^{T'''} = \frac{P(T'''|\mathbb{T} \leftarrow \mathbb{S})}{P(AnyOtherTestCode|\mathbb{T} \leftarrow \mathbb{S})}$$

QUESTION 2

- If TDD was hamstrung by keeping tests ahead of code, and needed support of a domain modelling technique when “designing” test cases, then what would that look like in the context of TDD?
- Especially if ASA means TDD plus BDD plus ATDD; and therefore, we need distinguish between D/G/R/S in prompts to set the correct abstraction for the testing.





THE PROPOSED CRC+STPA APPROACH

CRC+STPA IN A NUTSHELL

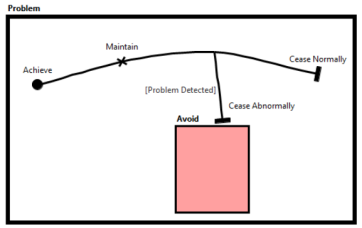
Table 1: Paradigm mapping

Intent Layer Concept	CRC	STPA
Goal allocation	Responsibility	Safety constraint
Behaviour coordination	Collaboration	Control structure
Behavioural influence	Notify	Control action
System state awareness	Shared collaboration context	Process model
Response propagation	Message passing	Feedback

- In principle there are difference between CRC and STPA in intended audiences.
- Table 1 looks at the paradigm mapping between the two methods. The only row of contention appears to be the “Goal Allocation” item, since the other rows relate to jargon from the domain for similar concepts (see next slide).
- The “Goal Allocation” row, however, can be deciphered by a GORE practitioner. Given Thomas describes the process of mitigation of a hazard as:

$$Requirement \longleftarrow Responsibility \longleftarrow SafetyConstraint \leftarrow Hazard$$

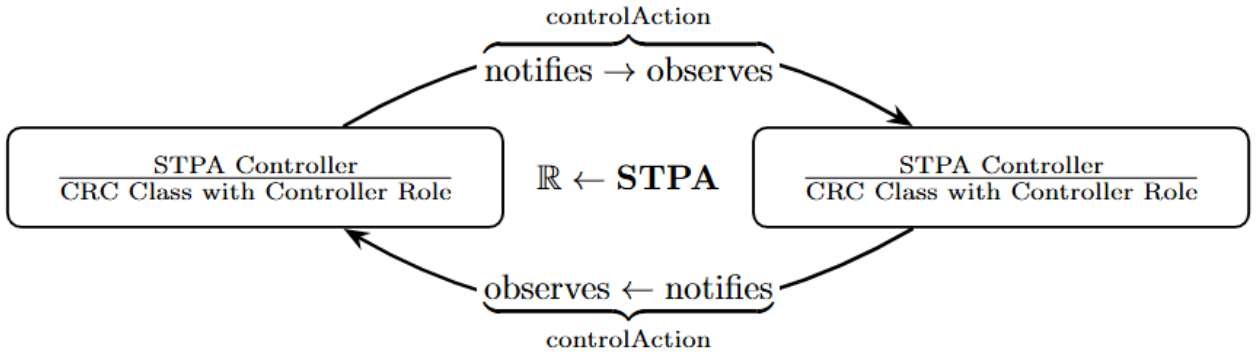
- The more interesting difference is therefore CRC has Responsibilities as first class element, whereas STPA does not.
- If the responsibilities are part of the reasoning chain, to avoid the bad outcome, so they ought to be captured as part of the analysis.



OR, …

Bump the design of the system in the Direction of Maximal Loss Reduction is the point of Thomas's:

$$Requirement \longleftarrow Responsibility \longleftarrow SafetyConstraint \leftarrow Hazard$$



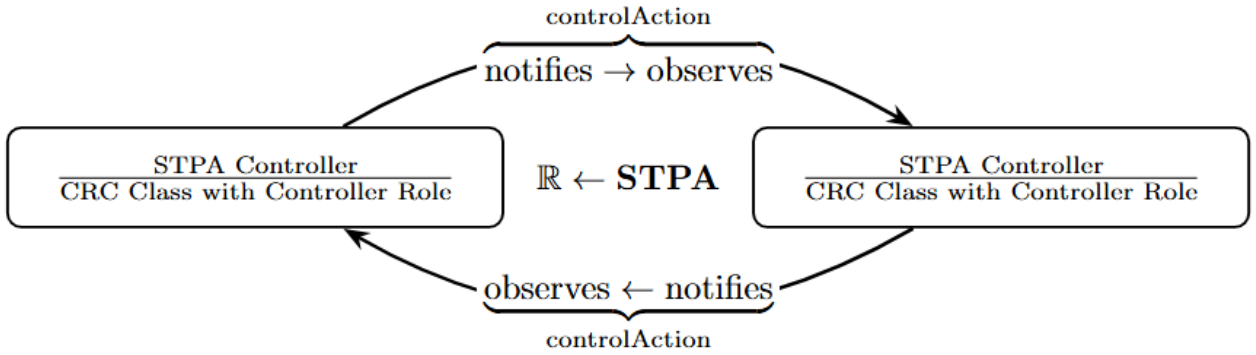
STPA is the lambda function that looks at generating a decreasing risk gradient ($-\nabla$), to avoid loss. STPA can be interpreted as achieving this by application of a rigorous means to trade off controller specification options (S) against a powerset of architectural requirements and their tensions (χ). The aim is to define a specification (S_o^c), on a Controller, that entails the intent of the allocation Responsibility ($\mathbb{R}$).

The lambda function meaning, analysts brings the tools.

OR, …

Bump the design of the system in the Direction of Maximal Loss Reduction is the point of Thomas's:

$$Requirement \longleftarrow Responsibility \longleftarrow SafetyConstraint \leftarrow Hazard$$



CRC has no mandated lambda function, to drive the risk gradient down. So why not use STPA as the basis.

CRC makes the Responsibilities first class elements to go towards assurance reasoning by way of rationale capture during the modelling.

CRC is also historically approachable by XP programmers.

OR, …

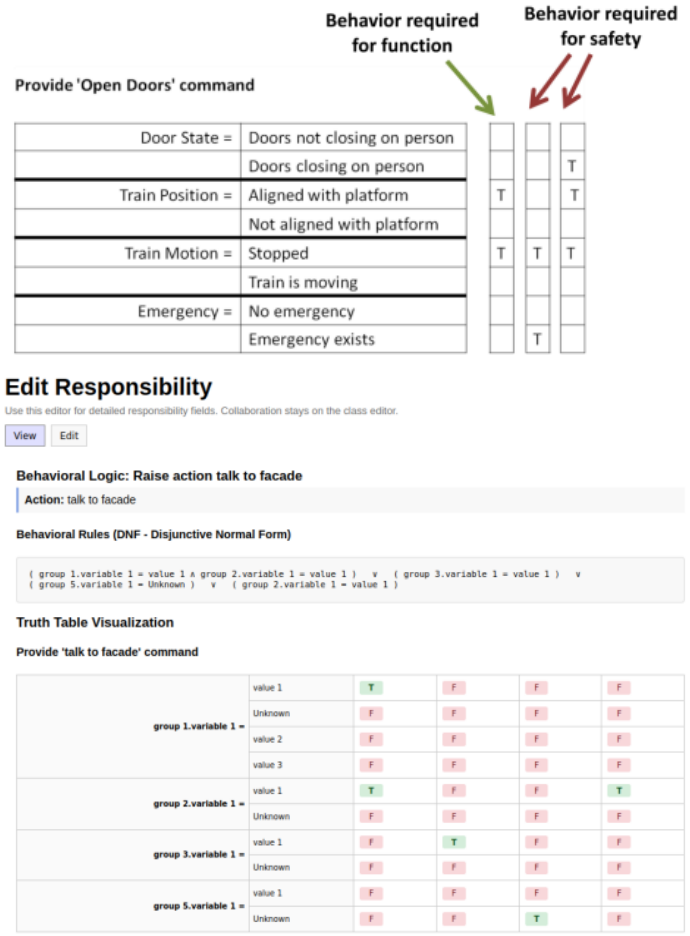
- **Provide a tool façade that is familiar to Agile programmers and has a “State Machine” abstraction (for both “happy” aka “White Hat” and “unhappy” aka “Black Hat paths).**





WORKED EXAMPLE

NOTHING NEW TO SEE HERE



Behavior required for function

Behavior required for safety

Provide 'Open Doors' command

Door State =	Doors not closing on person			
	Doors closing on person			T
Train Position =	Aligned with platform	T		T
	Not aligned with platform			
Train Motion =	Stopped	T	T	T
	Train is moving			
Emergency =	No emergency			
	Emergency exists		T	

Edit Responsibility

Use this editor for detailed responsibility fields. Collaboration stays on the class editor.

View | Edit

Behavioral Logic: Raise action talk to facade

Action: talk to facade

Behavioral Rules (DNF - Disjunctive Normal Form)

(group 1.variable 1 = value 1 ∧ group 2.variable 1 = value 1) ∨ (group 3.variable 1 = value 1) ∨ (group 5.variable 1 = Unknown) ∨ (group 2.variable 1 = value 1)

Truth Table Visualization

Provide 'talk to facade' command

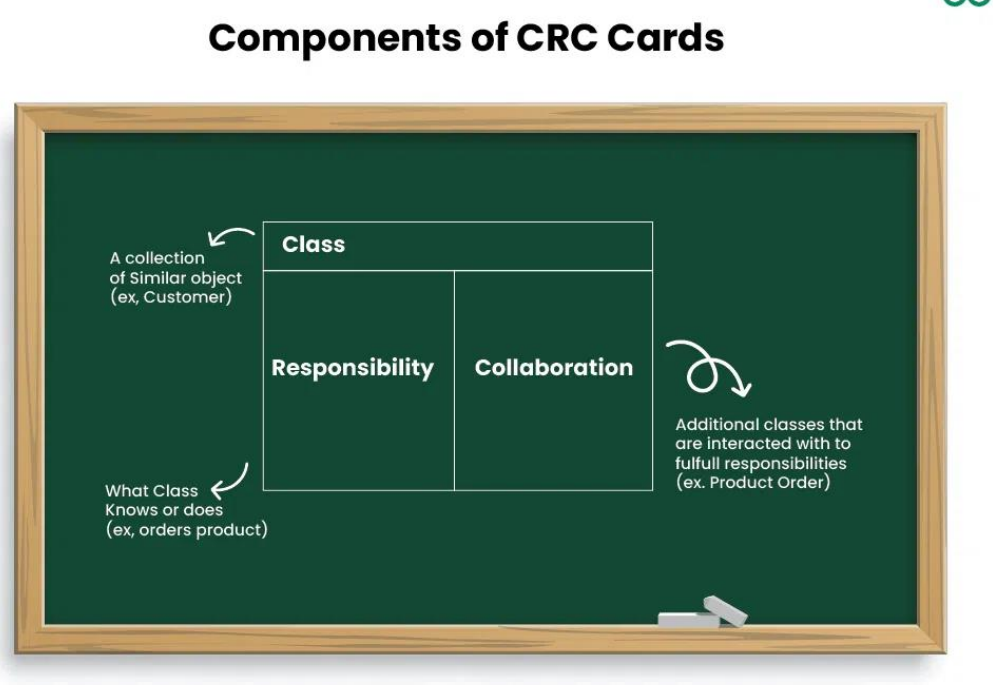
group 1.variable 1 =	value 1	T	F	F	F
	Unknown	F	F	F	F
	value 2	F	F	F	F
	value 3	F	F	F	F
group 2.variable 1 =	value 1	T	F	F	T
	Unknown	F	F	F	F
group 3.variable 1 =	value 1	F	T	F	F
	Unknown	F	F	F	F
group 5.variable 1 =	value 1	F	F	F	F
	Unknown	F	F	T	F

- Tool will essentially be an STPA model builder with responsibilities as first-class elements (care of CRC).
  - Tool is in work and is being refactored at the moment (having to undo architectural slop).
- The knowledge aspect of CRC formalised by using STPA Process Variables.
- The tool can currently define a range of responsibility types, especially notifies/observes pairs (aka Control Actions), and knowledge updates (aka process variable state changes).
- Sample bottom left is of a CRC behavioural responsibility that sends a notification (aka control action) to a collaboration class (both with role controller) per the Control Action table example top left. STPA jargon introduced in UI.

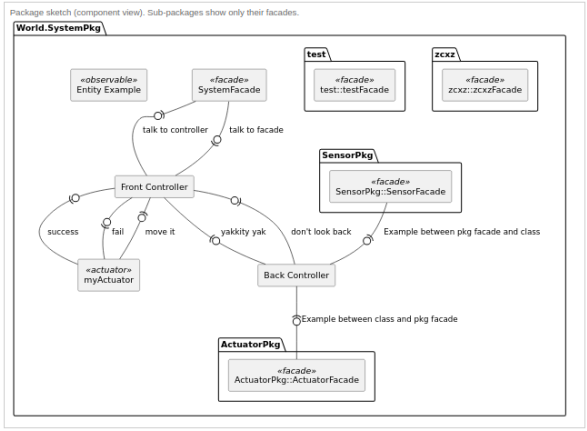
AND IT MASQUERADES AS A CRC TOOL

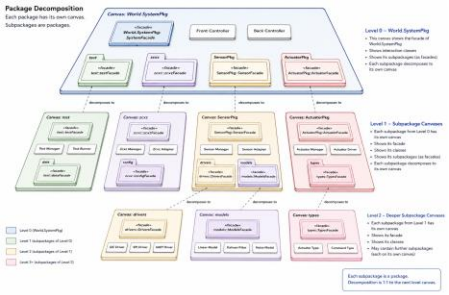
CRC Limitation	Semantic Resolution via STPA-Extended CRC++
Ambiguous collaborations	Mapped to guarded state transitions over *Process Variables*, providing explicit conditions under which interactions occur.
Informal responsibilities	Reinterpreted as invariants constrained by process-variable state, making obligations explicitly state-dependent.
Lack of runtime semantics	Resolved via explicit epistemic state transitions $K \rightarrow K'$, induced by observation events.
Weak causal structure	Strengthened through integration with STPA-style process-variable dependencies and control structure semantics.

With something extra under the hood



ADDITIONAL ENHANCEMENTS





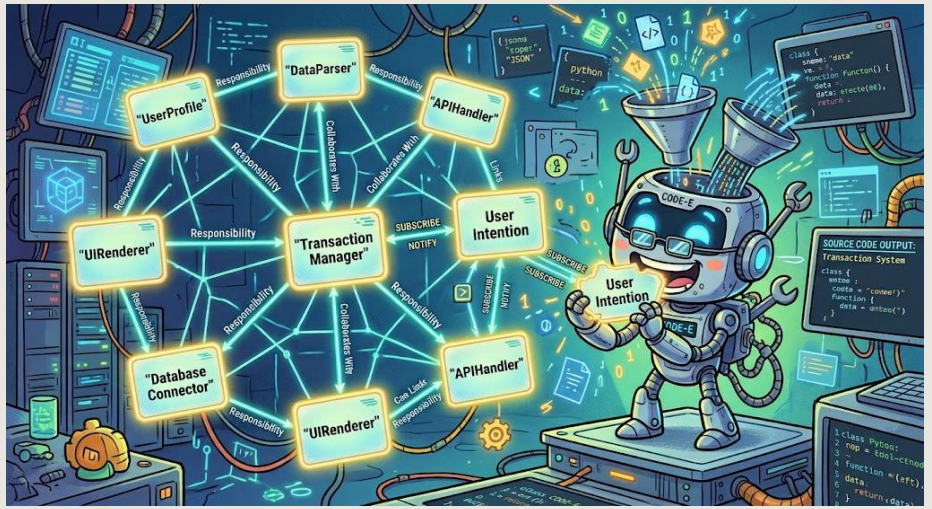
For each level $o \in [DomainAbstractionLevel]$, given $[P] \preccurlyeq [G] \preccurlyeq [R] \preccurlyeq [S]$ (so $\searrow$ side of the System-V):

$$\begin{aligned} \text{P: } & W_{[P]} \wedge G_{[P]} && \Rightarrow_B Solved(P); \\ \text{G: } & W_{[G]} \wedge R_{[G]} && \Rightarrow_B Realised(G); \\ \text{R: } & W_{[R]} \wedge S_{[R]} && \Rightarrow_B Operationalized(R); \\ \text{S: } & W_{[S]} \wedge S'''_{[S]} && \Rightarrow_B Encoded(S) \end{aligned}$$

- Thomas does not advocate structured top-down analysis, nor does CRC.
- However, since the role is to manage the complexity of the means-ends decomposition, and both requirement decomposition and refinement, a hierarchical model is used. The approach does this by adopting UML/SysML concept of packages.
- Package (and subpackage) canvases act as control diagrams. They are therefore familiar to domain modellers as well as STPA modellers.
- Package <<facades>> are a means to hide the complexity with sub-packages by forcing a Boundary object.
- Packaging manages the overall decomposition problem.



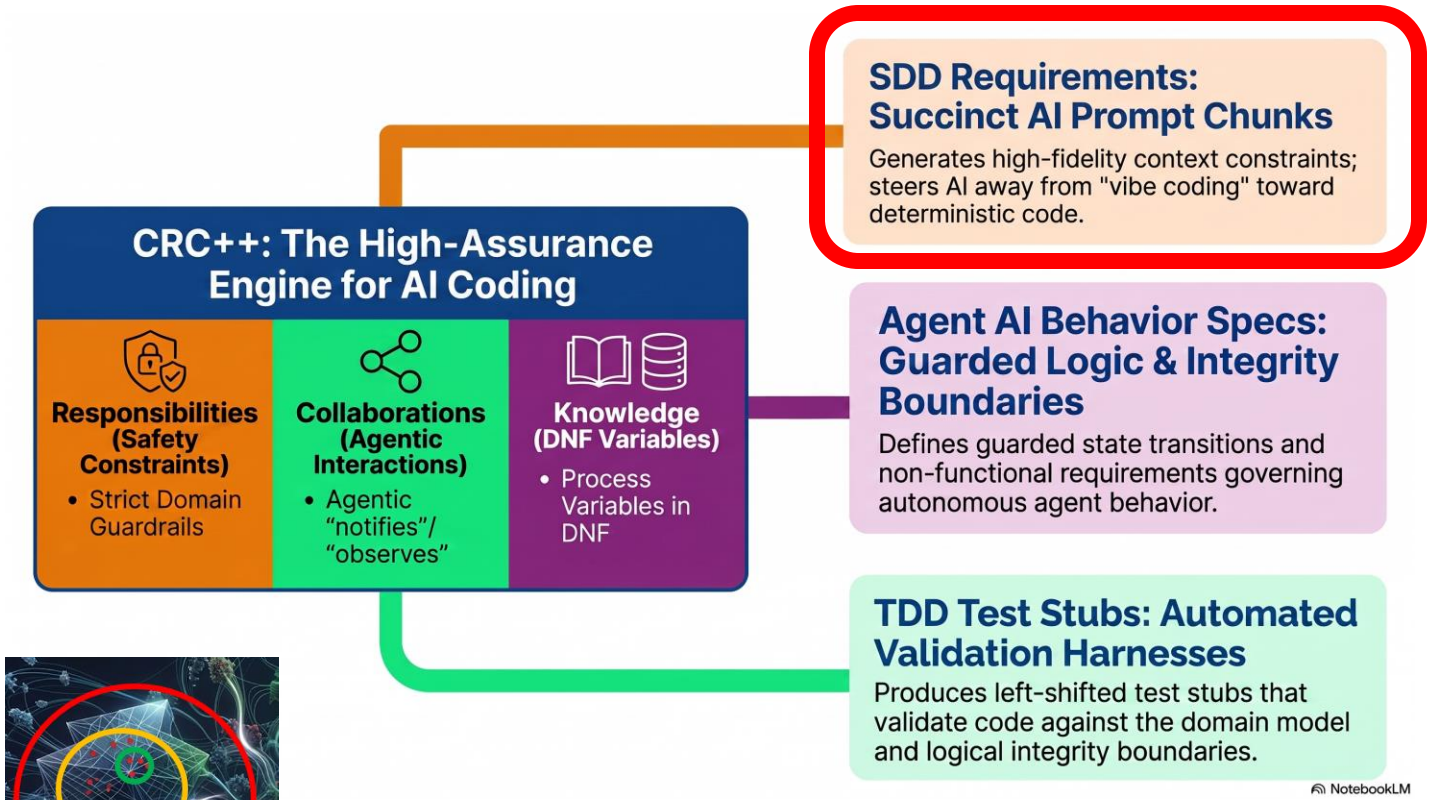
HOW WILL AI CODING AGENT CONSUME THE MODEL *

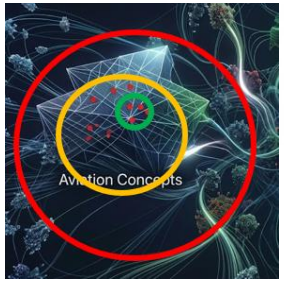


* Aspirational

REQUIREMENT MODELLING

- The tool is a spec modelling approach to constrain the requirement author into creating a domain model to ensure consistency and coherence in the behavioural specification as a whole.
- Export is to current SDD tools (Google SPEC Kit and Open Spec).
- Inclusion of requirement prompts and anti-requirement prompts both from the structured modelling.
- AI "reviewers" can be used to press for refinement of Requirements into more succinct Specifications; OR we can jump to code given we can already jump from "vibe" to code.

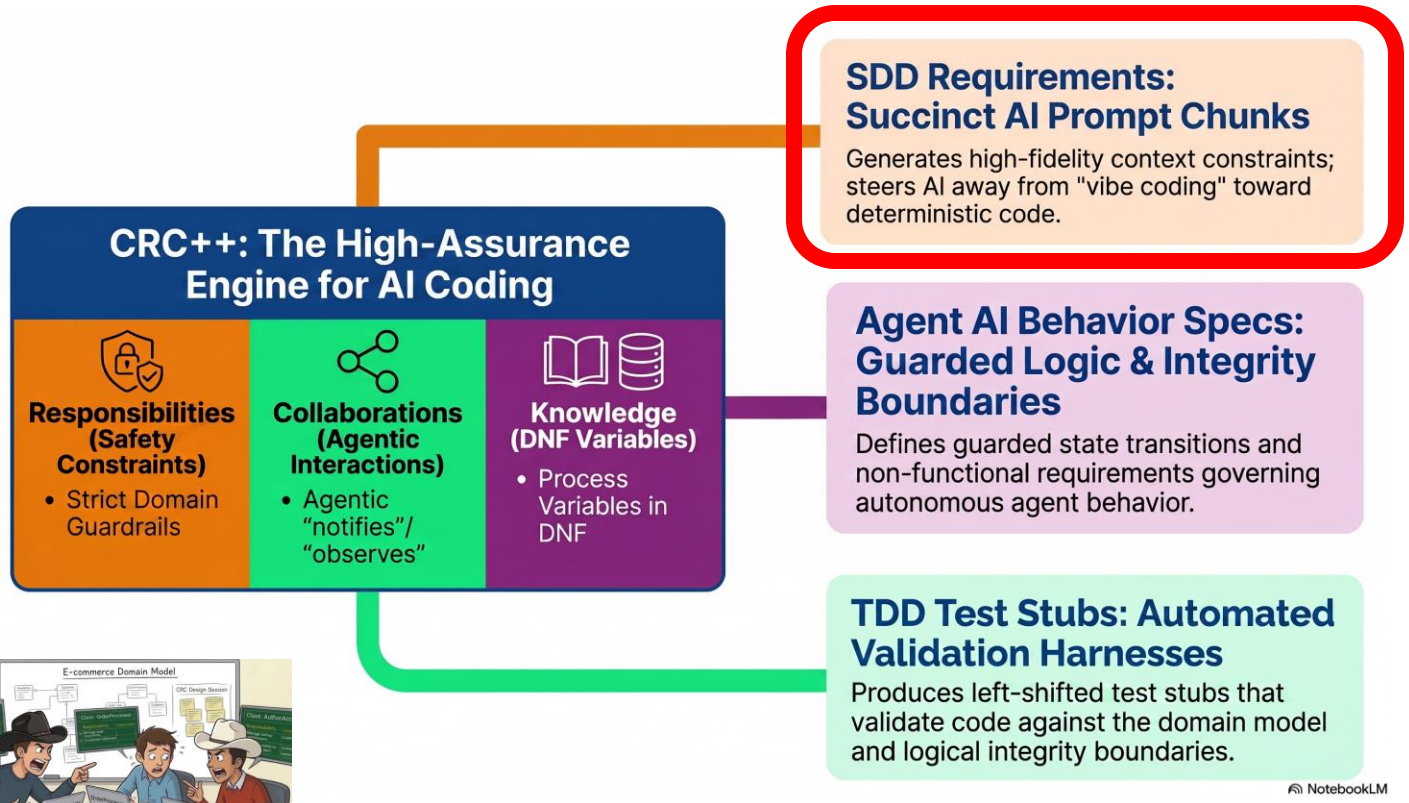


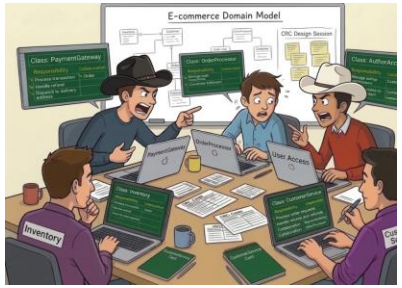


GAME ORIENTED SPECIFICATION MODELLING

- CRC is essentially a game. The tool is therefore aimed to be collaborative to allow multiple developers to participate.
- BLACK HAT and WHITE HAT developers prompting and anti-prompting at the olde Ok Corral.
- An AI agent might also “play” card roles potentially.

  * Aspirational

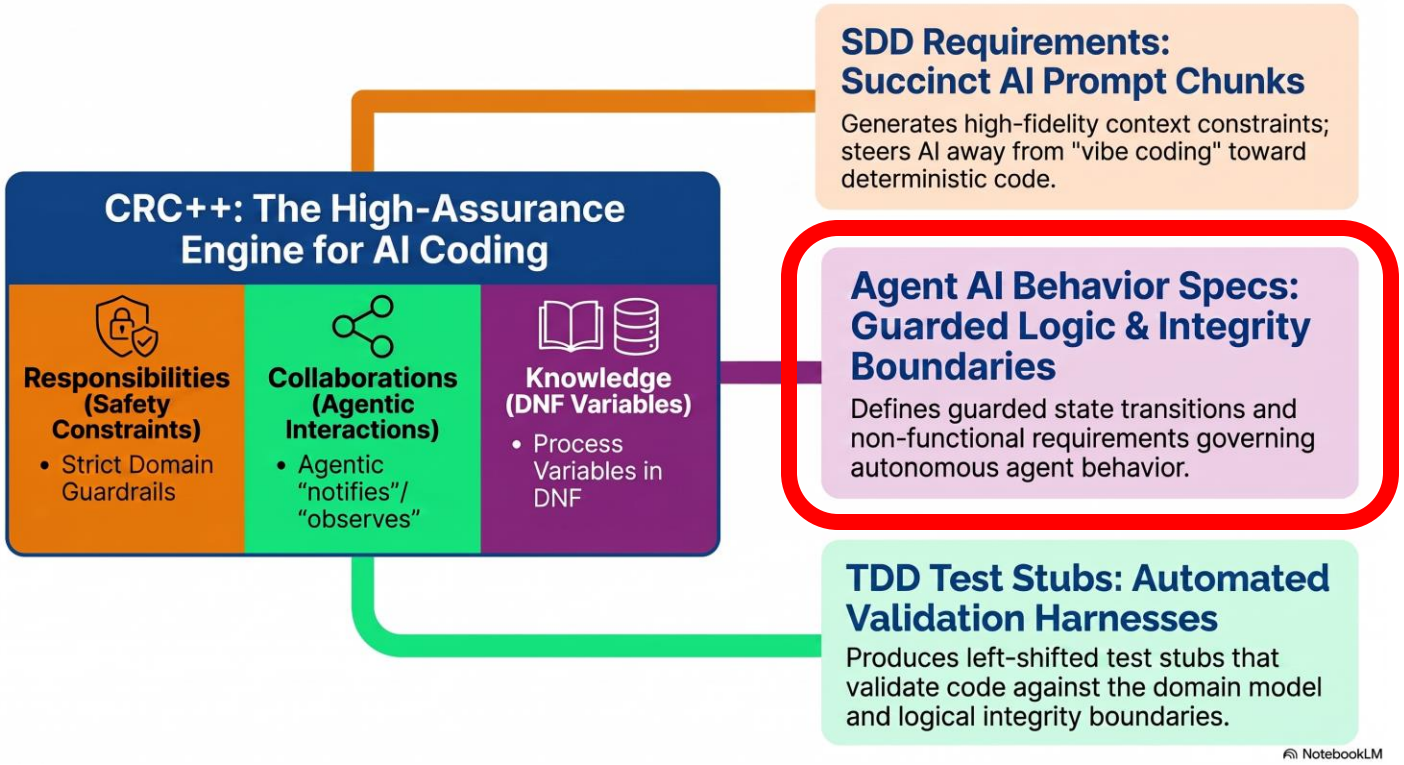




* Aspirational

AGENT ORIENTED ASPECTS

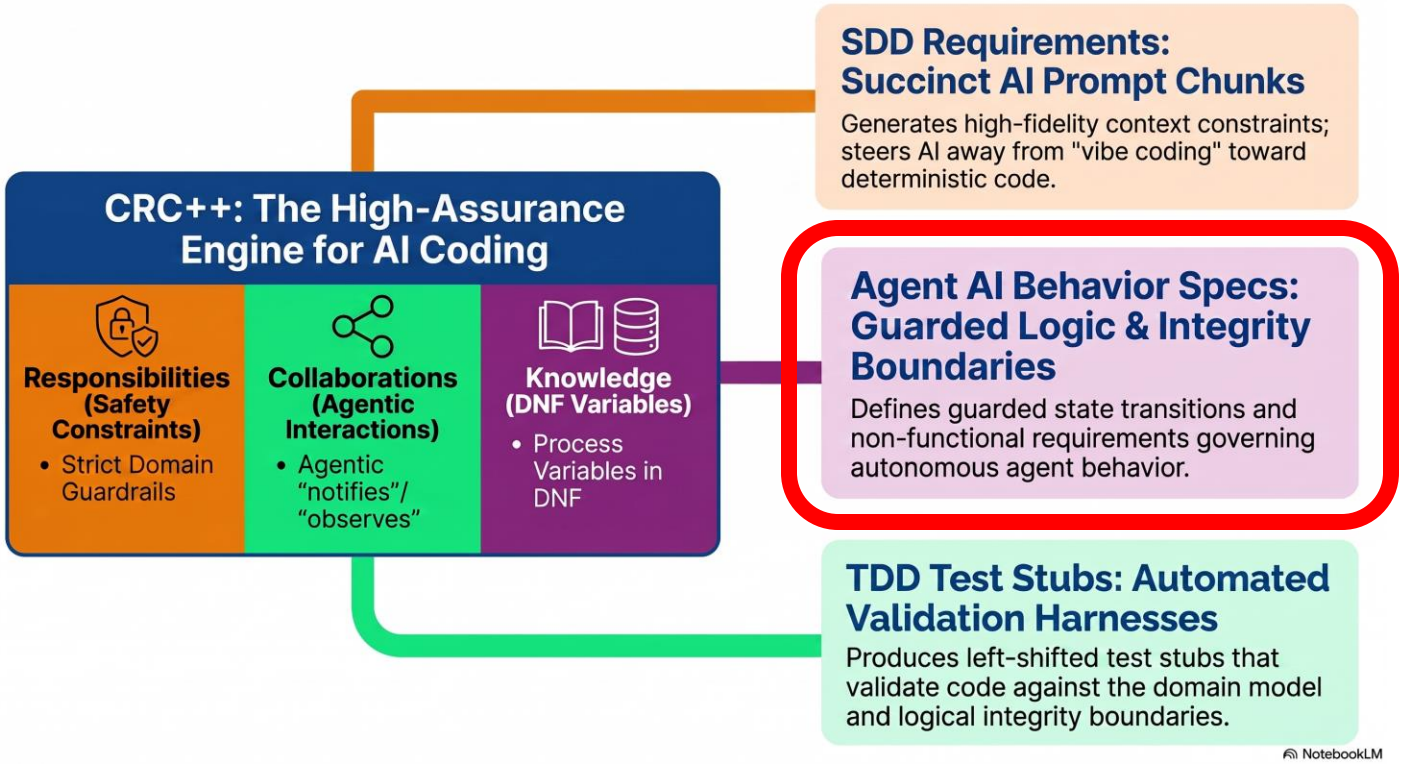
- AI Agents are Agent Oriented in the GORE sense.
- The behaviour spec aspects therefore lend themselves to specification of agent interactions.
- A2A is just a design note and the individual class “specs” modular agent prompts.



* Aspirational

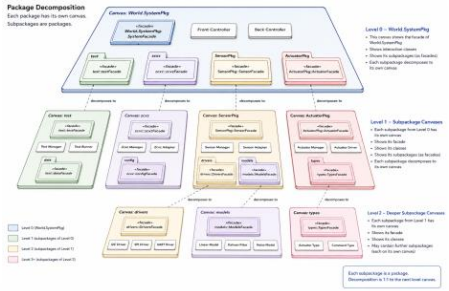
GAME ORIENTED CODING ASPECTS

- Given the agent-oriented specification style, it ought not be missed that coding agents could be allocated per class to avoid one coding agent having access to the entire code base.
- Orchestration would be by an Agent orchestration framework.
- The top-down means-end approach can be honoured by having MOCKING agents (playing the role of stubbed classes). As each level is coded the supporting levels can be MOCKED.

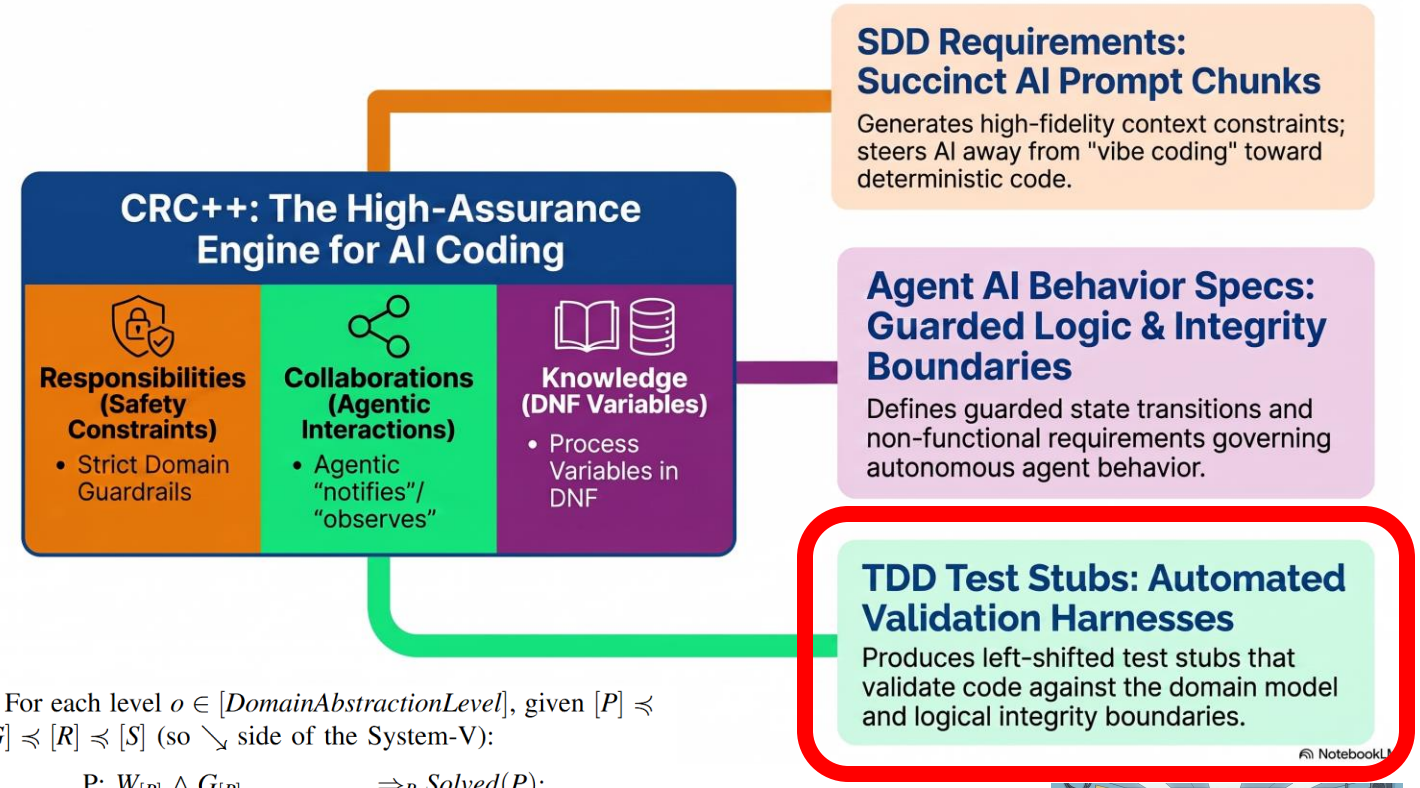


* Aspirational

TDD OR BDD OR ATDD

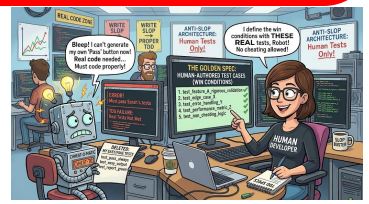


- Forward engineering of test cases from behaviour specs into a TDD, BDD or ATDD test abstraction BECAUSE of more succinctly distinguishing between desires, goals, requirements and specification.
- No AI intervention needed in the determination of level or test case specification.
  - AI might still be trusted to implement the test case.
- Anti-Requirements are obviously useful guards to be used in testing.

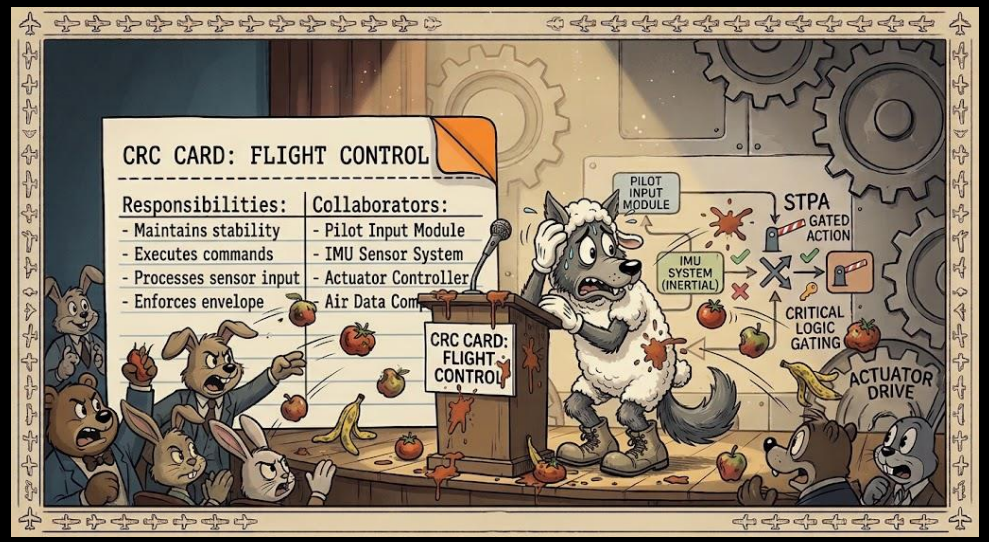


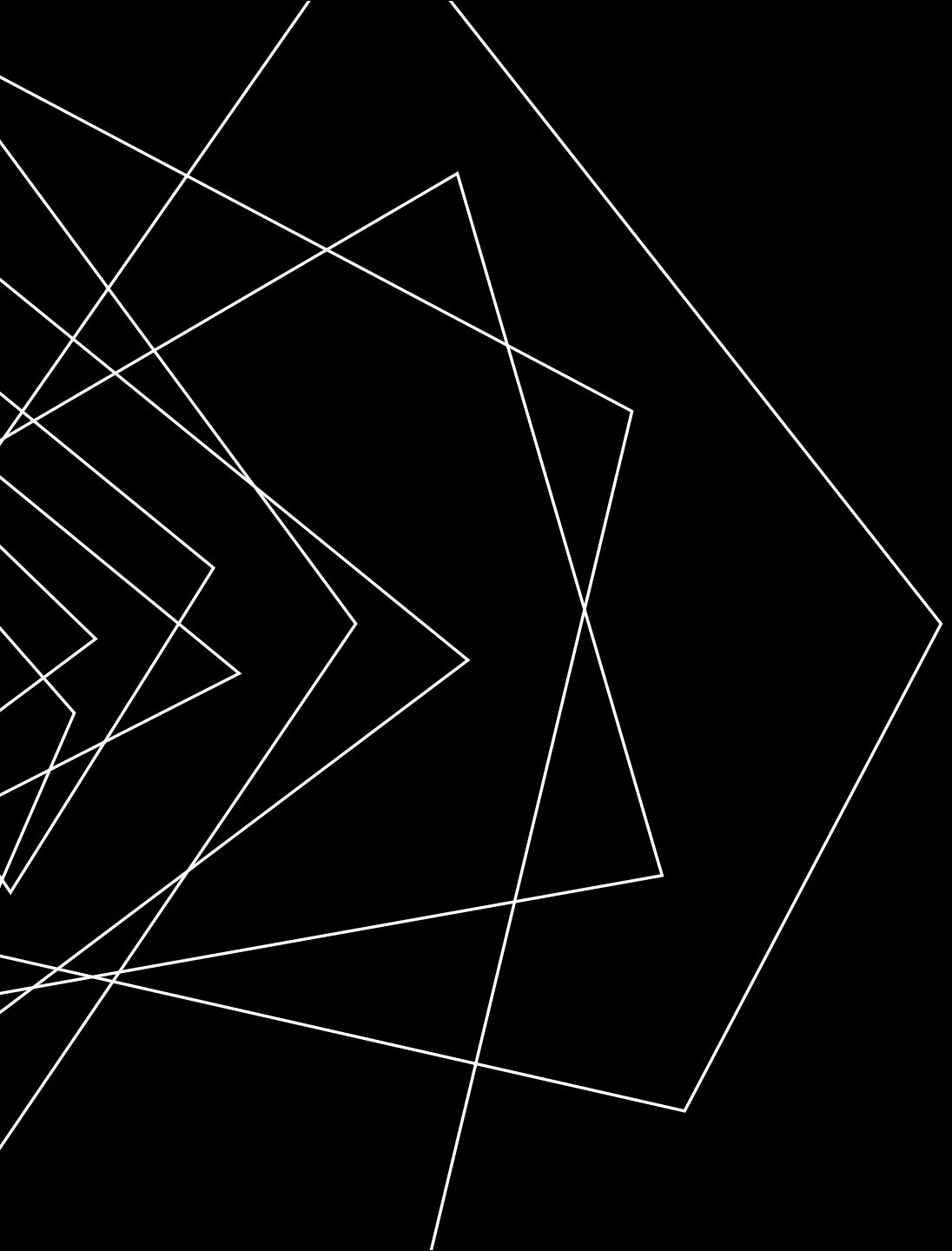
For each level $o \in [DomainAbstractionLevel]$, given $[P] \preccurlyeq [G] \preccurlyeq [R] \preccurlyeq [S]$ (so $\searrow$ side of the System-V):

$$
\begin{aligned}
\text{P: } & W_{[P]} \wedge G_{[P]} && \Rightarrow_B Solved(P); \\
\text{G: } & W_{[G]} \wedge R_{[G]} && \Rightarrow_B Realised(G); && \leftarrow ATDD \\
\text{R: } & W_{[R]} \wedge S_{[R]} && \Rightarrow_B Operationalized(R); && \leftarrow BDD \\
\text{S: } & W_{[S]} \wedge S'''_{[S]} && \Rightarrow_B Encoded(S) && \leftarrow TDD
\end{aligned}
$$



QUESTIONS


CRC CARD: FLIGHT CONTROL
Responsibilities:
- Maintains stability
- Executes commands
- Processes sensor input
- Enforces envelope
Collaborators:
- Pilot Input Module
- IMU Sensor System
- Actuator Controller
- Air Data Com
CRC CARD: FLIGHT CONTROL
PILOT INPUT MODULE
IMU SYSTEM (INERTIAL)
STPA
GATED ACTION
CRITICAL LOGIC GATING
ACTUATOR DRIVE



SPEC DRIVEN DEVELOPMENT SO FAR

Compelling the spec author to behave and manage complexity of specs across files versus a requirement spec model

EASY APPROACH TO REQUIREMENT SPECIFICATION

- System level statements
- Behavioural or assertions
- Unwanted behaviour
- Examples UI level

The 5 EARS Patterns

Pattern	Template Structure	Example
1. Ubiquitous	The `<system name>` shall `<system response>`.	The system shall log all user sign-in attempts.
2. State-Driven	While `<precondition>`, the `<system name>` shall `<system response>`.	While the aircraft is in flight, the system shall display cabin pressure.
3. Event-Driven	When `<trigger>`, the `<system name>` shall `<system response>`.	When the user clicks "Delete", the system shall prompt for confirmation.
4. Optional-Feature	Where `<optional feature is included>`, the `<system name>` shall `<system response>`.	Where the biometrics module is included, the system shall scan the user's fingerprint.
5. Unwanted-Behavior	If `<unwanted condition>`, then the `<system name>` shall `<system response>`.	If a network connection is lost, then the system shall buffer data locally.



WORKFLOW

- **Proposal – goal statement (optative)**
- **Context and action**
- **UI behaviour level**

- **Small Feature Path (`add-logout-button`)**
  - **Proposal (`proposal.md`)**: Defines *why* and *what* changes are happening (e.g., adding a logout action to clear user session).
  - **Scenario (`spec.md`)**:
    - **GIVEN** a user is logged into their account and viewing the header.
    - **WHEN** the user clicks the "Logout" button.
    - **THEN** the system SHALL clear the active session and show a confirmation dialog or redirect to the login screen.
  - **Tasks (`tasks.md`)**: 1.1 Add logout button to header, 1.2 Clear session on click, 1.3 Add confirmation dialog. GitHub

EMERGING DESIGN BY DESIGN

- Specification evolves versus a tension between emerging design and bug fixes
- "Domain-Specific" does not mean Domain-Modelling up front
- The example being one of two projects, between them explained as:
  - "not a toy snippet, but two full pretend products with a year of spec-driven history behind them"
- This in the face of "influencers" already demanding we don't need specs, just prototype. When the actuality SDD is spec driven prototyping.

- **Domain-Specific Examples (`openspec-examples`)**
  - Full reference repositories like wayfarer-bookings (a travel booking marketplace) and evergreen health systems demonstrate multi-month spec-driven git histories, ADRs, and playbook scenarios in practice. GitHub

- **Bug Fix Path (`fix-login-redirect-loop`)**
  - **Proposal (`proposal.md`)**: Captures the current broken behavior (infinite redirect loop on expired token) versus the intended behavior.
  - **Scenario (`spec.md`)**:
    - **GIVEN** a user has an expired authentication token.
    - **WHEN** the user attempts to access a protected route.
    - **THEN** the system SHALL redirect the user to `/login` exactly once without looping. GitHub

